CSE 3230 · LAB 4 HOME TASK

University Management System

Spring Boot REST API · Spring Security · Docker Compose

A complete study guide to every file and every piece of logic

Language / runtime	Java 21
Framework	Spring Boot 3.5.16 (Spring Framework 6.2, Spring Security 6.5, Hibernate 6.6)
Database	MySQL 8.4 (Docker) · H2 in-memory (tests only)
Containers	Dockerfile (multi-stage) + compose.yaml (app + db)
Browser testing	Swagger UI at `http://localhost:8080/swagger-ui.html`
Verified	16 automated tests pass · `demo.sh` and Swagger UI run against the real Docker stack

Read sections 1–4 for the big picture, then the section for each package. Section 19 lists 43 likely viva questions with answers.

Contents

1. The assignment and how each requirement is met
2. Technologies used and why
3. Project structure and the layered architecture
4. The life of one request, start to finish
5. Build file: pom.xml
6. Database design
7. model/ – the entities
8. repository/ – database access with Spring Data
9. dto/ – what the JSON looks like
10. service/ – business rules and transactions
11. controller/ – the REST endpoints
12. exception/ – consistent JSON errors
13. Spring Security – login, logout, hashing, RBAC
14. The first admin: config/DataInitializer
15. Configuration: application.yml
16. Docker and Docker Compose
17. Automated tests
18. Demonstrating it in the lab
19. Likely viva questions and answers
20. Known limitations and possible improvements

1. The assignment and how each requirement is met

Requirement	Where it is done	Proof
REST CRUD for Admin, Teacher, Student	`AdminController`, `TeacherController`, `StudentController` — each has GET all, GET by id, POST, PUT, DELETE	Test `fullCrudLifecycleForStudent`; demo steps 3 and 7
Proper REST conventions	Plural nouns in URLs (`/api/teachers/5`), HTTP method = action, no verbs in URLs, `Location` header on create	Section 11
JSON responses	Records in `dto/` turned to JSON by Jackson; errors use one JSON shape (`ApiError`), including security errors	Sections 9, 12
HTTP status codes	200, 201, 204, 400, 401, 403, 404, 409 (table in section 12)	Tests check each code
Login	`POST /api/auth/login` in `AuthController` (JSON body → session cookie). HTTP Basic also accepted.	Tests `loginWithSeededAdmin...`, `httpBasic...`
Logout	Spring Security `LogoutFilter` at `POST /api/auth/logout`, configured in `SecurityConfig`	Test `logoutInvalidatesSession`
Password hashing	`BCryptPasswordEncoder` bean; `UserAccountService.applyCommonFields` hashes before saving	Test `passwordIsStoredAsBcryptHash...`; MySQL shows `$2a$10$...`
Role-based access control	`authorizeHttpRequests` rules in `SecurityConfig`	Tests `adminCan...`, `teacherCan...`, `studentCan...`
Admin manages all three; teacher only teachers; student only students	`/api/admins/**` → ADMIN · `/api/teachers/**` → ADMIN, TEACHER · `/api/students/**` → ADMIN, STUDENT	Demo steps 5, 6
Dockerfile	Two-stage build: Maven builds the jar, a small JRE image runs it as a non-root user	Section 16
compose.yaml	Two services, `db` (MySQL with health check and a volume) and `app` (waits for a healthy db)	`docker compose up --build`
Extra: try it in a browser	Swagger UI lists every endpoint with an Execute button; supports both login methods	Section 18.2

Extra things done that a teacher might ask about

- **Validation** of every request body (empty name, bad email, etc. → 400 with a message per field).
- **Unique** usernames and emails across all three roles, and unique student IDs (→ 409 Conflict).
- **First admin is created automatically** on startup, otherwise nobody could log in (only admins can create admins).
- **The last admin cannot be deleted** (→ 409), so the system can never lock itself out.

- **Password never appears in any response**, because responses are built from DTOs that have no password field.
- **Swagger UI**: an interactive web page to log in and call every endpoint from the browser (no Postman needed).
- **Automated tests** (16) that go through the real security filters, controllers, services and a database.

2. Technologies used and why

Technology	What it does here	Why this choice
Java 21	Programming language	Current long-term-support (LTS) release; gives us `record`, text blocks, `.toList()`
Spring Boot 3.5.16	Starts an embedded Tomcat web server, wires all objects together, reads configuration	Latest patch of the mature 3.x line. Spring Boot 4 exists but is newer with breaking changes; 3.5 has the most tutorials/documentation for a lab
Spring Web (MVC)	`@RestController`, URL mapping, JSON in/out via Jackson	Standard way to build REST APIs in Spring
Spring Data JPA + Hibernate	Maps Java classes to tables, generates SQL	We write almost no SQL; repositories are interfaces only
MySQL 8.4	Real database inside Docker	8.4 is MySQL's LTS line; widely used in courses
Spring Security 6.5	Authentication (who are you), authorization (what may you do), BCrypt	The standard security framework for Spring
Jakarta Bean Validation	`@NotBlank`, `@Email`, `@Size` on request fields	Declarative validation, no if-statements in controllers
H2	In-memory database for tests	Tests run anywhere in seconds without MySQL
JUnit 5 + MockMvc	Automated tests sending fake HTTP requests through the whole app	Included in `spring-boot-starter-test`
springdoc-openapi 2.8.17 (Swagger UI)	Reads our controllers and generates an OpenAPI description + an interactive web page	Lets anyone try the API from a browser; the 2.8.x line is the one built for Spring Boot 3.5
Docker + Docker Compose	Package the app as an image; run app + MySQL together with one command	Same environment on every machine; the assignment requires it
Maven + wrapper (`mvnw`)	Build tool: downloads libraries, compiles, tests, packages a jar	The wrapper downloads the right Maven version, so nobody needs Maven installed

3. Project structure and the layered architecture

```
university-management-system/
├── pom.xml                          Maven build file: dependencies + Java version
├── mvnw, mvnw.cmd, .mvn/            Maven wrapper (run ./mvnw instead of mvn)
├── Dockerfile                       How to build the app's Docker image
├── compose.yaml                     Runs app + MySQL together
├── .env.example                     Template for passwords used by compose
├── .dockerignore, .gitignore        Files Docker / Git should ignore
├── demo.sh                          curl walkthrough of every feature (for the lab demo)
├── README.md                        Short how-to-run
├── docs/UMS-Explained.pdf           This guide
└── src/
    ├── main/java/com/university/ums/
    │   ├── UmsApplication.java      main() - starts everything
    │   ├── model/                   JPA entities = database tables
    │   │   ├── Role.java                ADMIN / TEACHER / STUDENT
    │   │   ├── AppUser.java             common fields (abstract parent)
    │   │   ├── Admin.java, Teacher.java, Student.java
    │   ├── repository/              database access (Spring Data interfaces)
    │   ├── dto/                     JSON shapes for requests and responses
    │   ├── service/                 business rules + transactions
    │   ├── controller/              HTTP endpoints
    │   ├── security/                Spring Security configuration
    │   ├── exception/               error types + global JSON error handler
    │   └── config/                  DataInitializer (first admin), OpenApiConfig (Swagger UI)
    ├── main/resources/application.yml    configuration (DB, session cookie, first admin)
    ├── test/java/.../UniversityApiTests.java   15 end-to-end tests
    └── test/resources/application.yml    test config (H2 database)
```

Why split code into layers?

Each layer has one job and only talks to the layer below it. This is called **separation of concerns**. It makes each class small, easy to test, and easy to explain:

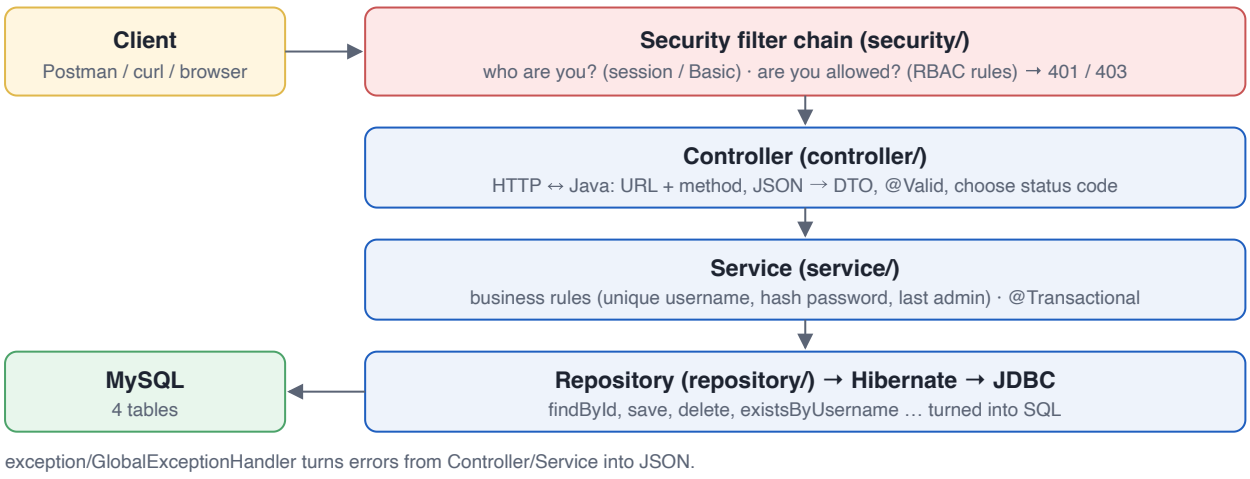


Figure 1 – Layers. A request goes down; the response comes back up the same way. dto/ records carry the data between client, controller and service.

Layer	Knows about	Does NOT know about
Controller	HTTP (URLs, status codes, JSON), DTOs, services	SQL, how passwords are hashed
Service	Business rules, entities, repositories, transactions	HTTP (no request/response objects)
Repository	Entities and database queries	Business rules
Model (entity)	Columns and tables	Everything else

Key Spring ideas used everywhere

- **Bean**: an object that Spring creates and manages. Classes annotated with `@Service`, `@RestController`, `@Component`, `@Configuration`, and methods annotated with `@Bean`, all produce beans.
- **Dependency Injection (DI)**: a class does not create its helpers with `new`; it lists them as constructor parameters and Spring passes them in. Example: `TeacherService(TeacherRepository teacherRepository, UserAccountService accounts)`. Benefits: loose coupling, easy testing, one shared instance (singleton).
- **Constructor injection** (used in every class here) is preferred over `@Autowired` on fields because fields can be `final`, dependencies are obvious, and the object can never exist half-initialised. With a single constructor Spring injects automatically, no annotation needed.
- **Component scanning**: `@SpringBootApplication` scans the package `com.university.ums` and all sub-packages for annotated classes. That is why every class lives under that package.
- **Auto-configuration**: Spring Boot sees a library on the classpath (e.g. MySQL driver, Spring Security) and configures it with sensible defaults (DataSource, Jackson ObjectMapper, a security filter chain...). We only override what we need.

4. The life of one request, start to finish

Example: a logged-in **teacher** sends `POST /api/teachers` with a JSON body to create another teacher.

1. **Tomcat** (the web server embedded in the jar, port 8080) receives the HTTP request and hands it to the servlet filter chain.
2. **Spring Security filters** run first (details in section 13):
   - `SecurityContextHolderFilter` reads the `JSESSIONID` cookie, finds the HTTP session on the server and loads the saved `SecurityContext` (it holds the teacher's `Authentication` with authority `ROLE_TEACHER`).
   - `LogoutFilter`: is this `POST /api/auth/logout`? No → continue.
   - `BasicAuthenticationFilter`: is there an `Authorization: Basic` header? No → continue.
   - `AuthorizationFilter` checks the rules top to bottom. `/api/teachers/**` requires ADMIN or TEACHER. User has ROLE_TEACHER → allowed.
3. **DispatcherServlet** (Spring MVC's front controller) finds the method mapped to `POST /api/teachers`: `TeacherController.create`.
4. **Jackson** converts the JSON body into a `TeacherRequest` record (`@RequestBody`). Bad JSON → 400.

5. **Bean Validation** checks the annotations on the record (`@Valid`). Any violation → `MethodArgumentNotValidException` → 400 with field errors.
6. **Controller** calls `teacherService.create(request)`.
7. **Service** starts a database transaction (`@Transactional`), checks a password was given, checks the username/email are not taken (→ 409), builds a `Teacher` entity, hashes the password with BCrypt and calls `teacherRepository.save(teacher)`.
8. **Hibernate** runs `INSERT INTO users (...)` then `INSERT INTO teachers (department, designation, id)`. The transaction commits.
9. The service converts the saved entity to a `TeacherResponse` (no password) and returns it.
10. The **controller** builds the URL of the new resource (`http://localhost:8080/api/teachers/7`) and returns `201 Created` with a `Location` header and the JSON body.
11. **Jackson** turns the response record into JSON; Tomcat sends it to the client.

> If anything throws an exception in steps 6–9, `GlobalExceptionHandler` catches it and returns a JSON error. If the role check fails in step 2, the controller is never reached; `JsonSecurityHandlers` writes the 401/403 JSON instead.

5. Build file: pom.xml

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- Spring Boot parent: manages versions of every Spring dependency and plugin -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.16</version>
        <relativePath/>
    </parent>

    <groupId>com.university</groupId>
    <artifactId>university-management-system</artifactId>
    <version>1.0.0</version>
    <name>University Management System</name>
    <description>CSE 3230 Lab 4 - REST API with Spring Security and Docker</description>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <!-- REST controllers, embedded Tomcat, Jackson (JSON) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <!-- JPA + Hibernate for talking to the database -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```xml
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <!-- Login, logout, password hashing, role-based access control -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <!-- @NotBlank, @Email, @Size ... on request bodies -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <!-- Swagger UI at /swagger-ui.html: browse and call every endpoint from the browser -->
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.17</version>
        </dependency>
        <!-- MySQL JDBC driver (used in Docker / production) -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- In-memory database used only by the automated tests -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- Packages the app as one executable "fat" jar: java -jar app.jar -->
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

- **parent spring-boot-starter-parent 3.5.16**: we inherit tested versions for every Spring library and plugin. That is why the dependencies below have no `<version>`. It also enables the compiler flag `-parameters`, which lets `@PathVariable Long id` work without writing `@PathVariable("id")`.
- **java.version 21**: compile for Java 21.
- **Starters** are bundles: `spring-boot-starter-web` brings Spring MVC + Tomcat + Jackson; `-data-jpa` brings Hibernate + Spring Data; `-security` brings Spring Security; `-validation` brings Hibernate Validator.

- **springdoc-openapi-starter-webmvc-ui** is the only dependency with an explicit `<version>`: it is not a Spring project, so the Spring Boot parent does not manage its version.
- **Scopes**: `runtime` (MySQL driver) = not needed to compile, needed to run. `test` (H2, test libraries) = only on the test classpath, not inside the final jar.
- **spring-boot-maven-plugin** repackages the build into an executable "fat jar" containing our classes and every library, so `java -jar app.jar` runs it.

6. Database design

All three roles share many fields (name, email, phone, username, password). Repeating them in three unrelated tables would mean three separate username lists: a teacher and a student could both pick `rahim`, and login would not know whom you mean. So the design uses **inheritance**: an abstract parent entity `AppUser` with three child entities.

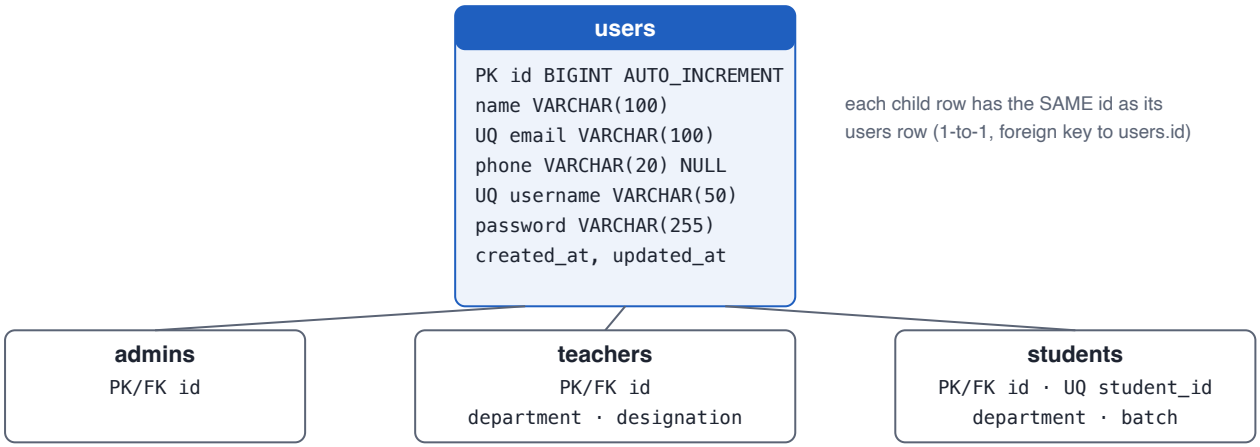


Figure 2 – Tables generated by Hibernate (JOINED inheritance). PK = primary key, FK = foreign key, UQ = unique.

The actual tables in MySQL (copied from `SHOW CREATE TABLE`)

```
CREATE TABLE `users` (
  `id` bigint NOT NULL AUTO_INCREMENT,
  `created_at` datetime(6) NOT NULL,
  `email` varchar(100) NOT NULL,
  `name` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `phone` varchar(20) DEFAULT NULL,
  `updated_at` datetime(6) NOT NULL,
  `username` varchar(50) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `UK6dotkott2kjsp8vw4d0m25fb7` (`email`),
  UNIQUE KEY `UKr43af9ap4edm43mmtq01oddj6` (`username`)
)
CREATE TABLE `teachers` (
  `department` varchar(100) NOT NULL,
  `designation` varchar(50) NOT NULL,
  `id` bigint NOT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `FKpavufmal5lbtc60csriy8sx3` FOREIGN KEY (`id`) REFERENCES `users` (`id`)
)
CREATE TABLE `students` (
  `batch` int NOT NULL,
  `department` varchar(100) NOT NULL,
  `student_id` varchar(20) NOT NULL,
  `id` bigint NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `UK5mbus2m1tm2acucrp6t627jmx` (`student_id`),
  CONSTRAINT `FK7xqmtv7r2eb5axni3jm0a80su` FOREIGN KEY (`id`) REFERENCES `users` (`id`)
)
CREATE TABLE `admins` (
  `id` bigint NOT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `FKanhsicqm3lc8ya77tr7r0je18` FOREIGN KEY (`id`) REFERENCES `users` (`id`)
)
```

Notice: Java field `studentId` became column `student_id`, and `createdAt` became `created_at`. Spring Boot's default naming strategy converts camelCase to snake_case. The odd names like `UKr43af...` are constraint names Hibernate generates automatically.

SQL that Hibernate generates

Java call	SQL (simplified)
`teacherRepository.save(newTeacher)`	`INSERT INTO users(name,email,...) VALUES(...)`; then `INSERT INTO teachers(department,designation,id) VALUES(?,?,<new id>)`
`teacherRepository.findAll()`	`SELECT ... FROM teachers t JOIN users u ON u.id = t.id`
`userRepository.findByUsername("karim")`	`SELECT ... FROM users u LEFT JOIN admins a ON .. LEFT JOIN teachers t ON .. LEFT JOIN students s ON .. WHERE u.username=?`; Hibernate checks which child table had a row and creates a `Teacher` object (polymorphism)
`teacherRepository.delete(t)`	`DELETE FROM teachers WHERE id=?`; then `DELETE FROM users WHERE id=?`

Why JOINED and not the other inheritance strategies?

Strategy	Tables	Good	Bad
JOINED (used)	users + one per child	Normalised, no NULL columns, NOT NULL constraints on child columns, one unique username list	Reads need a JOIN
SINGLE_TABLE	one big table + a "type" column	Fastest, no joins	Child columns must allow NULL (a student row has no designation) → weaker data integrity
TABLE_PER_CLASS	three full tables	No joins for one type	Username uniqueness across roles cannot be enforced by the database; IDs must be shared across tables
No inheritance	three unrelated tables	Simple	Duplicated code; login has to search three tables; duplicate usernames possible

7. model/ – the entities

An **entity** is a Java class whose objects are stored as rows in a table. JPA (Jakarta Persistence API) is the standard that defines annotations like `@Entity`; **Hibernate** is the library that implements it and writes the SQL.

src/main/java/com/university/ums/model/Role.java

```java
package com.university.ums.model;

public enum Role {
    ADMIN,
    TEACHER,
    STUDENT
}
```

An `enum` is a fixed set of constants. Using an enum instead of strings like `"teacher"` means typos are compile errors. `Role.TEACHER.name()` returns the text `"TEACHER"`, which the security layer turns into the authority `ROLE_TEACHER`.

src/main/java/com/university/ums/model/AppUser.java

```java
package com.university.ums.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Inheritance;
import jakarta.persistence.InheritanceType;
import jakarta.persistence.PrePersist;
import jakarta.persistence.PreUpdate;
import jakarta.persistence.Table;

import java.time.Instant;

/**
 * Fields shared by every person who can log in (admin, teacher, student).
 * JOINED inheritance: common columns live in "users", role-specific columns
 * live in "admins" / "teachers" / "students", linked by the same id.
 * One "users" table means usernames and emails are unique across all roles.
 */
@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(length = 20)
    private String phone;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    // BCrypt hash, never the plain-text password
    @Column(nullable = false)
    private String password;

    @Column(nullable = false, updatable = false)
    private Instant createdAt;
```

```java

    @Column(nullable = false)
    private Instant updatedAt;

    @PrePersist
    void onCreate() {
        createdAt = Instant.now();
        updatedAt = createdAt;
    }

    @PreUpdate
    void onUpdate() {
        updatedAt = Instant.now();
    }

    /** Each subclass says which role it is; Spring Security turns this into ROLE_xxx. */
    public abstract Role getRole();

    public Long getId() { return id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public Instant getCreatedAt() { return createdAt; }
    public Instant getUpdatedAt() { return updatedAt; }
}
```

Line-by-line logic

- `@Entity`: this class is mapped to a table. `@Table(name = "users")`: the table name. We avoid `user` because it is a reserved word in some databases.
- `@Inheritance(strategy = InheritanceType.JOINED)`: children get their own tables joined by id (section 6).
- `abstract class`: you can never create a plain `AppUser`; every user must be an Admin, Teacher or Student. The name is `AppUser`, not `User`, so it does not clash with Spring Security's own `User` class.
- `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)`: primary key generated by the database's `AUTO_INCREMENT`. Note there is **no setId()**: clients and our code can never choose an id.
- `@Column(nullable = false, unique = true, length = 50)`: becomes `NOT NULL`, a `UNIQUE` index and `VARCHAR(50)`. This is the last line of defence: even if the Java check is skipped, the database refuses duplicates.
- `password`: holds the BCrypt hash (60 characters), never the real password. Default length 255 is plenty.
- `createdAt` has `updatable = false`: Hibernate never includes it in an UPDATE, so it can't change after insert.
- `@PrePersist onCreate()`: a **JPA lifecycle callback**. Hibernate calls it automatically just before the INSERT, setting both timestamps. `@PreUpdate onUpdate()` runs just before every UPDATE and

refreshes `updatedAt`. No service code has to remember to set times.

- `public abstract Role getRole();`: each child returns its own role. Because it is a method, not a field, there is no `role` column — the role is implied by which child table has the row, so it can never contradict the data.
- Getters/setters: JPA and Jackson need them to read/write fields. Plain Java is used instead of Lombok so there is no hidden generated code to explain.

src/main/java/com/university/ums/model/Admin.java

```java
package com.university.ums.model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;

@Entity
@Table(name = "admins")
public class Admin extends AppUser {

    @Override
    public Role getRole() {
        return Role.ADMIN;
    }
}
```

Admin adds no extra columns; it only says "I am an ADMIN". Its table `admins` just contains the id. That row is what marks a user as an admin. JPA needs a no-argument constructor; Java provides one automatically because no other constructor is written.

src/main/java/com/university/ums/model/Teacher.java

```java
package com.university.ums.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Table;

@Entity
@Table(name = "teachers")
public class Teacher extends AppUser {

    @Column(nullable = false, length = 100)
    private String department;

    // e.g. Lecturer, Assistant Professor, Professor
    @Column(nullable = false, length = 50)
    private String designation;

    @Override
    public Role getRole() {
        return Role.TEACHER;
    }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }

    public String getDesignation() { return designation; }
    public void setDesignation(String designation) { this.designation = designation; }
}
```

src/main/java/com/university/ums/model/Student.java

```java
package com.university.ums.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Table;

@Entity
@Table(name = "students")
public class Student extends AppUser {

    // University roll / registration number, e.g. "2007001"
    @Column(nullable = false, unique = true, length = 20)
    private String studentId;

    @Column(nullable = false, length = 100)
    private String department;

    // Admission year, e.g. 2021
    @Column(nullable = false)
    private Integer batch;

    @Override
    public Role getRole() {
        return Role.STUDENT;
    }

    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId = studentId; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }

    public Integer getBatch() { return batch; }
    public void setBatch(Integer batch) { this.batch = batch; }
}
```

`extends AppUser` means a Teacher *is an* AppUser: it inherits id, name, email, username, password, timestamps. It adds department and designation. Student adds `studentId` (roll number, unique), department and batch (admission year as an `Integer`; the wrapper type is used so "missing" can be detected as `null` during validation).

8. repository/ – database access with Spring Data

src/main/java/com/university/ums/repository/UserRepository.java

```java
package com.university.ums.repository;

import com.university.ums.model.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

/** Queries across all three roles at once (the shared "users" table). */
public interface UserRepository extends JpaRepository<AppUser, Long> {

    Optional<AppUser> findByUsername(String username);

    boolean existsByUsername(String username);

    boolean existsByEmail(String email);

    // "...AndIdNot": ignore the user being updated, so keeping your own username is not a conflict
    boolean existsByUsernameAndIdNot(String username, Long id);

    boolean existsByEmailAndIdNot(String email, Long id);
}
```

src/main/java/com/university/ums/repository/TeacherRepository.java

```java
package com.university.ums.repository;

import com.university.ums.model.Teacher;
import org.springframework.data.jpa.repository.JpaRepository;

public interface TeacherRepository extends JpaRepository<Teacher, Long> {
}
```

src/main/java/com/university/ums/repository/StudentRepository.java

```java
package com.university.ums.repository;

import com.university.ums.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Long> {

    boolean existsByStudentId(String studentId);

    boolean existsByStudentIdAndIdNot(String studentId, Long id);
}
```

(`AdminRepository` is identical to `TeacherRepository` with `Admin`.)

How can an interface with no code work?

At startup Spring Data creates a class implementing each interface (a **proxy**) and registers it as a bean. `JpaRepository<Teacher, Long>` means "entity type Teacher, id type Long" and gives us, for free:

Method	What it does
`save(entity)`	INSERT if new, UPDATE (merge) if it already has an id
`saveAndFlush(entity)`	save and immediately send the SQL to the database
`findById(id)`	returns `Optional<Teacher>` (empty if not found, never null)
`findAll()`, `count()`, `delete(entity)`, `existsById(id)`	as the names say

Derived queries: the method name IS the query

Spring Data parses the method name and writes the query:

Method name	Generated JPQL / SQL meaning
`findByUsername(String u)`	`SELECT u FROM AppUser u WHERE u.username = ?1`
`existsByEmail(String e)`	"is there at least one row WHERE email = ?" → `true/false`
`existsByUsernameAndIdNot(String u, Long id)`	`WHERE username = ?1 AND id <> ?2`
`existsByStudentIdAndIdNot(String s, Long id)`	`WHERE student_id = ?1 AND id <> ?2`

> **Why "AndIdNot"?** When teacher #5 is updated and keeps the username `karim`, a plain `existsByUsername("karim")` would find *teacher #5 itself* and wrongly report a conflict. Excluding the current id asks "does *someone else* already have this username?".

`UserRepository` works on the parent type `AppUser`, so its queries search **all** roles at once. That is what makes usernames and emails unique across admins, teachers and students, and what login uses.

9. dto/ – what the JSON looks like

A **DTO** (Data Transfer Object) is a class whose only job is to carry data in or out of the API. We never send or receive entities directly. Reasons:

1. **Security – no password leak.** Response DTOs simply have no password field. Returning the `Teacher` entity would print the hash.
2. **Security – no mass assignment.** Request DTOs contain only the fields a client may set. A client cannot send `"id": 1`, `"role": "ADMIN"` or `"createdAt"` to overwrite them.
3. **Validation** rules belong to the API input, so they live on the request DTO.
4. **Freedom to change** the database without changing the API (and vice versa).

Java records

`public record TeacherRequest(String name, ...) {}` is a short way to write an immutable data class. Java generates the constructor, the accessor methods `name()`, `email()`..., `equals`, `hashCode` and `toString`. Jackson knows how to read JSON into records and write records as JSON.

src/main/java/com/university/ums/dto/TeacherRequest.java

```java
package com.university.ums.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

/**
 * JSON body for POST /api/teachers and PUT /api/teachers/{id}.
 * password is required on create, optional on update (omit it to keep the old one).
 */
public record TeacherRequest(
        @NotBlank @Size(max = 100) String name,
        @NotBlank @Email @Size(max = 100) String email,
        @Size(max = 20) String phone,
        @NotBlank @Size(min = 3, max = 50)
        @Pattern(regexp = "^[A-Za-z0-9._-]+$", message = "may contain only letters, digits, '.',
'_' and '-'")
        String username,
        @Size(min = 6, max = 72) String password,
        @NotBlank @Size(max = 100) String department,
        @NotBlank @Size(max = 50) String designation
) {
}
```

Validation annotations used

Annotation	Rule	Error message if broken
`@NotBlank`	not null, not "" and not only spaces (Strings)	must not be blank
`@NotNull`	not null (used for `Integer batch`)	must not be null
`@Email`	looks like an email address	must be a well-formed email address
`@Size(min, max)`	length limits; match the column lengths so the DB never rejects a long value	size must be between …
`@Pattern(regexp=…)`	username only letters, digits, `.` `_` `-` (no spaces)	our custom message
`@Min(1950)` `@Max(2100)`	batch year range	must be greater than or equal to 1950

These annotations do nothing by themselves; they are checked when the controller parameter is marked `@Valid`. Annotations on `null` values pass (except `@NotNull/@NotBlank`), which is why `phone` and `password` are optional.

> **Password rule:** `@Size(min = 6, max = 72)` but no `@NotBlank`, because the same DTO is used for create and update. On **create**, the service demands it (`requirePassword` → 400). On **update**, leaving it out means "keep the current password". The maximum 72 is because BCrypt only uses the first 72 bytes of a password.

src/main/java/com/university/ums/dto/TeacherResponse.java

```java
package com.university.ums.dto;

import com.university.ums.model.Role;
import com.university.ums.model.Teacher;

import java.time.Instant;

public record TeacherResponse(
        Long id,
        String name,
        String email,
        String phone,
        String username,
        Role role,
        String department,
        String designation,
        Instant createdAt,
        Instant updatedAt
) {
    public static TeacherResponse from(Teacher teacher) {
        return new TeacherResponse(
                teacher.getId(),
                teacher.getName(),
                teacher.getEmail(),
                teacher.getPhone(),
                teacher.getUsername(),
                teacher.getRole(),
                teacher.getDepartment(),
                teacher.getDesignation(),
                teacher.getCreatedAt(),
                teacher.getUpdatedAt());
    }
}
```

The `static from(Teacher)` method is a **factory/mapper**: it copies fields from the entity into the record. Every service method returns DTOs, so the controller never touches an entity. `AdminRequest/Response` and `StudentRequest/Response` follow the same pattern with their own fields:

src/main/java/com/university/ums/dto/StudentRequest.java

```java
package com.university.ums.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

/**
 * JSON body for POST /api/students and PUT /api/students/{id}.
 * password is required on create, optional on update (omit it to keep the old one).
 */
public record StudentRequest(
        @NotBlank @Size(max = 100) String name,
        @NotBlank @Email @Size(max = 100) String email,
        @Size(max = 20) String phone,
        @NotBlank @Size(min = 3, max = 50)
        @Pattern(regexp = "^[A-Za-z0-9._-]+$", message = "may contain only letters, digits, '.', '_' and '-'")
        String username,
        @Size(min = 6, max = 72) String password,
        @NotBlank @Size(max = 20) String studentId,
        @NotBlank @Size(max = 100) String department,
        @NotNull @Min(1950) @Max(2100) Integer batch
) {
}
```

src/main/java/com/university/ums/dto/LoginRequest.java

```java
package com.university.ums.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;

public record LoginRequest(
        // example values pre-fill the Swagger UI form
        @Schema(example = "admin") @NotBlank String username,
        @Schema(example = "admin123") @NotBlank String password
) {
}
```

`@Schema(example = ...)` is only documentation for Swagger UI: it pre-fills the login form with the seeded admin. It has no effect on validation or security.

src/main/java/com/university/ums/dto/CurrentUserResponse.java

```java
package com.university.ums.dto;

import com.university.ums.model.AppUser;
import com.university.ums.model.Role;

/** Returned by /api/auth/login and /api/auth/me: who is logged in and with which role. */
public record CurrentUserResponse(
        Long id,
        String name,
        String username,
        Role role
) {
    public static CurrentUserResponse from(AppUser user) {
        return new CurrentUserResponse(user.getId(), user.getName(), user.getUsername(),
user.getRole());
    }
}
```

src/main/java/com/university/ums/dto/MessageResponse.java

```java
package com.university.ums.dto;

public record MessageResponse(String message) {
}
```

Example JSON in and out

```
POST /api/students
{"name":"Rahim","email":"rahim@uni.edu","username":"rahim","password":"secret123",
 "studentId":"2007001","department":"CSE","batch":2020}

201 Created     Location: http://localhost:8080/api/students/3
{"id":3,"name":"Rahim","email":"rahim@uni.edu","phone":null,"username":"rahim","role":"STUDENT",
 "studentId":"2007001","department":"CSE","batch":2020,
 "createdAt":"2026-10-24T19:24:50.326420343Z","updatedAt":"2026-10-24T19:24:50.326420343Z"}
```

Times are `Instant` values (a moment in UTC); Spring Boot's Jackson setup prints them in ISO-8601 format (the trailing `Z` means UTC).

10. service/ – business rules and transactions

10.1 UserAccountService – shared account logic

src/main/java/com/university/ums/service/UserAccountService.java

```java
package com.university.ums.service;

import com.university.ums.exception.BadRequestException;
import com.university.ums.exception.ConflictException;
import com.university.ums.model.AppUser;
import com.university.ums.repository.UserRepository;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.Locale;

/** Login-account logic shared by the admin, teacher and student services. */
```

```java
@Service
public class UserAccountService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public UserAccountService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    /** Usernames and emails are stored lower-case so "Rahim" and "rahim" are the same account. */
    public static String normalize(String value) {
        return value.trim().toLowerCase(Locale.ROOT);
    }

    public void requirePassword(String rawPassword) {
        if (rawPassword == null || rawPassword.isBlank()) {
            throw new BadRequestException("Password is required when creating a user");
        }
    }

    /**
     * Throws 409 if the username or email already belongs to someone else (in any role).
     * currentUserId is null on create, or the id of the user being updated.
     */
    public void ensureUnique(String username, String email, Long currentUserId) {
        String normalizedUsername = normalize(username);
        String normalizedEmail = normalize(email);

        boolean usernameTaken = currentUserId == null
                ? userRepository.existsByUsername(normalizedUsername)
                : userRepository.existsByUsernameAndIdNot(normalizedUsername, currentUserId);
        if (usernameTaken) {
            throw new ConflictException("Username '" + normalizedUsername + "' is already taken");
        }

        boolean emailTaken = currentUserId == null
                ? userRepository.existsByEmail(normalizedEmail)
                : userRepository.existsByEmailAndIdNot(normalizedEmail, currentUserId);
        if (emailTaken) {
            throw new ConflictException("Email '" + normalizedEmail + "' is already in use");
        }
    }

    /** Copies the common fields onto the entity. The password is hashed with BCrypt, never stored raw. */
    public void applyCommonFields(AppUser user, String name, String email, String phone,
                                  String username, String rawPassword) {
        user.setName(name.trim());
        user.setEmail(normalize(email));
        user.setPhone(phone == null || phone.isBlank() ? null : phone.trim());
        user.setUsername(normalize(username));
        if (rawPassword != null && !rawPassword.isBlank()) {
            user.setPassword(passwordEncoder.encode(rawPassword));
        }
    }
}
```

- **Why a shared service?** All three roles need the same three things: uniqueness checks, field copying and password hashing. Writing them once avoids three copies that could drift apart.
- `normalize()`: trims spaces and lower-cases. `Locale.ROOT` makes lower-casing behave the same on every computer (in a Turkish locale, `"I".toLowerCase()` is not `"i"`). Result: `Rahim`, `RAHIM` and

`rahim` are the same account, on MySQL (case-insensitive by default) and H2 (case-sensitive) alike.

- `requirePassword()`: on create, a missing/blank password → `BadRequestException` → 400.
- `ensureUnique(username, email, currentUserId)`: `currentUserId == null` means "creating"; use `existsByUsername`. Otherwise "updating"; use `existsByUsernameAndIdNot` so the user's own current username is not a conflict. The ternary operator `condition ? a : b` chooses between the two queries. A clash throws `ConflictException` → 409.
- `applyCommonFields()`: copies trimmed/normalised values onto any `AppUser` (works for Admin, Teacher and Student because of inheritance). A blank phone is stored as `null`. **The password is only changed when a new one is supplied, and it is passed through** `passwordEncoder.encode()`, which returns a BCrypt hash. The raw password is never stored anywhere.

10.2 AdminService

src/main/java/com/university/ums/service/AdminService.java

```java
package com.university.ums.service;

import com.university.ums.dto.AdminRequest;
import com.university.ums.dto.AdminResponse;
import com.university.ums.exception.ConflictException;
import com.university.ums.exception.ResourceNotFoundException;
import com.university.ums.model.Admin;
import com.university.ums.repository.AdminRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@Transactional
public class AdminService {

    private final AdminRepository adminRepository;
    private final UserAccountService accounts;

    public AdminService(AdminRepository adminRepository, UserAccountService accounts) {
        this.adminRepository = adminRepository;
        this.accounts = accounts;
    }

    @Transactional(readOnly = true)
    public List<AdminResponse> findAll() {
        return adminRepository.findAll().stream().map(AdminResponse::from).toList();
    }

    @Transactional(readOnly = true)
    public AdminResponse findById(Long id) {
        return AdminResponse.from(getOrThrow(id));
    }

    public AdminResponse create(AdminRequest request) {
        accounts.requirePassword(request.password());
        accounts.ensureUnique(request.username(), request.email(), null);

        Admin admin = new Admin();
        apply(admin, request);
        return AdminResponse.from(adminRepository.save(admin));
    }

    public AdminResponse update(Long id, AdminRequest request) {
```

```
        Admin admin = getOrThrow(id);
        accounts.ensureUnique(request.username(), request.email(), id);

        apply(admin, request);
        // saveAndFlush runs the UPDATE now, so the response already shows the new updatedAt
        return AdminResponse.from(adminRepository.saveAndFlush(admin));
    }

    public void delete(Long id) {
        Admin admin = getOrThrow(id);
        // Without at least one admin nobody could ever create admins again
        if (adminRepository.count() <= 1) {
            throw new ConflictException("Cannot delete the last admin account");
        }
        adminRepository.delete(admin);
    }

    private void apply(Admin admin, AdminRequest request) {
        accounts.applyCommonFields(admin, request.name(), request.email(), request.phone(),
                request.username(), request.password());
    }

    private Admin getOrThrow(Long id) {
        return adminRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Admin", id));
    }
}
```

@Transactional – what it does

Putting `@Transactional` on the class makes every public method run inside a **database transaction**: Spring opens a transaction before the method, **commits** if it returns normally and **rolls back** if it throws a `RuntimeException`. So a method either completes fully or changes nothing (atomicity). Spring does this by wrapping the bean in a proxy object that starts/ends the transaction around the call.

`@Transactional(readOnly = true)` on the find methods tells Hibernate it does not need to track changes, which is a small optimisation and documents intent.

Method by method

- `findAll()`: load all admins, map each with `AdminResponse::from` (a method reference, same as `a -> AdminResponse.from(a)`), collect to a list. Streams: `.stream().map(...).toList()`.
- `findById(id)` → `getOrThrow(id)`: `findById` returns an `Optional`; `orElseThrow(() -> new ResourceNotFoundException("Admin", id))` unwraps it or throws → 404. The lambda means the exception is only created if needed.
- `create()`: order matters: (1) password present? (2) username/email free? Only then (3) build the entity, (4) copy fields + hash, (5) `save`, (6) convert to DTO. Checks come first so nothing is written when the request is invalid.
- `update()`: load the existing admin (404 if missing), check uniqueness excluding itself, copy the new values. The entity is **managed** (loaded in this transaction), so Hibernate would detect the changes and run UPDATE at commit by itself (**dirty checking**). We call `saveAndFlush` to run the UPDATE *now*: that triggers `@PreUpdate`, so the returned JSON already shows the new `updatedAt`.
- `delete()`: 404 if not found. **Last-admin rule:** if there is only one admin left, refuse with 409. Only admins can create admins, so deleting the last one would make admin features unreachable forever.

10.3 TeacherService and StudentService

src/main/java/com/university/ums/service/TeacherService.java

```java
package com.university.ums.service;

import com.university.ums.dto.TeacherRequest;
import com.university.ums.dto.TeacherResponse;
import com.university.ums.exception.ResourceNotFoundException;
import com.university.ums.model.Teacher;
import com.university.ums.repository.TeacherRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@Transactional
public class TeacherService {

    private final TeacherRepository teacherRepository;
    private final UserAccountService accounts;

    public TeacherService(TeacherRepository teacherRepository, UserAccountService accounts) {
        this.teacherRepository = teacherRepository;
        this.accounts = accounts;
    }

    @Transactional(readOnly = true)
    public List<TeacherResponse> findAll() {
        return teacherRepository.findAll().stream().map(TeacherResponse::from).toList();
    }

    @Transactional(readOnly = true)
    public TeacherResponse findById(Long id) {
        return TeacherResponse.from(getOrThrow(id));
    }

    public TeacherResponse create(TeacherRequest request) {
        accounts.requirePassword(request.password());
        accounts.ensureUnique(request.username(), request.email(), null);

        Teacher teacher = new Teacher();
        apply(teacher, request);
        return TeacherResponse.from(teacherRepository.save(teacher));
    }

    public TeacherResponse update(Long id, TeacherRequest request) {
        Teacher teacher = getOrThrow(id);
        accounts.ensureUnique(request.username(), request.email(), id);

        apply(teacher, request);
        return TeacherResponse.from(teacherRepository.saveAndFlush(teacher));
    }

    public void delete(Long id) {
        teacherRepository.delete(getOrThrow(id));
    }

    private void apply(Teacher teacher, TeacherRequest request) {
        accounts.applyCommonFields(teacher, request.name(), request.email(), request.phone(),
                request.username(), request.password());
        teacher.setDepartment(request.department().trim());
        teacher.setDesignation(request.designation().trim());
    }
```

```
62
63      private Teacher getOrThrow(Long id) {
64          return teacherRepository.findById(id)
65                  .orElseThrow(() -> new ResourceNotFoundException("Teacher", id));
66      }
67  }
```

src/main/java/com/university/ums/service/StudentService.java

```
1   package com.university.ums.service;
2
3   import com.university.ums.dto.StudentRequest;
4   import com.university.ums.dto.StudentResponse;
5   import com.university.ums.exception.ConflictException;
6   import com.university.ums.exception.ResourceNotFoundException;
7   import com.university.ums.model.Student;
8   import com.university.ums.repository.StudentRepository;
9   import org.springframework.stereotype.Service;
10  import org.springframework.transaction.annotation.Transactional;
11
12  import java.util.List;
13
14  @Service
15  @Transactional
16  public class StudentService {
17
18      private final StudentRepository studentRepository;
19      private final UserAccountService accounts;
20
21      public StudentService(StudentRepository studentRepository, UserAccountService accounts) {
22          this.studentRepository = studentRepository;
23          this.accounts = accounts;
24      }
25
26      @Transactional(readOnly = true)
27      public List<StudentResponse> findAll() {
28          return studentRepository.findAll().stream().map(StudentResponse::from).toList();
29      }
30
31      @Transactional(readOnly = true)
32      public StudentResponse findById(Long id) {
33          return StudentResponse.from(getOrThrow(id));
34      }
35
36      public StudentResponse create(StudentRequest request) {
37          accounts.requirePassword(request.password());
38          accounts.ensureUnique(request.username(), request.email(), null);
39          if (studentRepository.existsByStudentId(request.studentId().trim())) {
40              throw new ConflictException("Student ID '" + request.studentId().trim() + "' is already
registered");
41          }
42
43          Student student = new Student();
44          apply(student, request);
45          return StudentResponse.from(studentRepository.save(student));
46      }
47
48      public StudentResponse update(Long id, StudentRequest request) {
49          Student student = getOrThrow(id);
50          accounts.ensureUnique(request.username(), request.email(), id);
51          if (studentRepository.existsByStudentIdAndIdNot(request.studentId().trim(), id)) {
52              throw new ConflictException("Student ID '" + request.studentId().trim() + "' is already
registered");
53          }
54
```

```
55          apply(student, request);
56          return StudentResponse.from(studentRepository.saveAndFlush(student));
57      }
58
59      public void delete(Long id) {
60          studentRepository.delete(getOrThrow(id));
61      }
62
63      private void apply(Student student, StudentRequest request) {
64          accounts.applyCommonFields(student, request.name(), request.email(), request.phone(),
65                  request.username(), request.password());
66          student.setStudentId(request.studentId().trim());
67          student.setDepartment(request.department().trim());
68          student.setBatch(request.batch());
69      }
70
71      private Student getOrThrow(Long id) {
72          return studentRepository.findById(id)
73                  .orElseThrow(() -> new ResourceNotFoundException("Student", id));
74      }
75  }
```

Same structure as AdminService. Differences: the private `apply()` also copies the role-specific fields; StudentService additionally checks that `studentId` (roll number) is not already registered to another student (409). Teachers and students have no "last one" rule.

> **Why check uniqueness in Java when the database also has UNIQUE constraints?** The Java check gives a clear message ("Username 'rahim' is already taken"). The database constraint protects against a **race condition**: if two requests with the same username arrive at the same instant, both Java checks may pass, but the database will reject the second INSERT. That exception (`DataIntegrityViolationException`) is turned into a 409 by the global handler. Both layers together = friendly *and* safe.

11. controller/ – the REST endpoints

REST conventions followed

Action	Method + URL	Success code	Body	Idempotent?
List all	`GET /api/teachers`	200 OK	JSON array	yes (read only)
Read one	`GET /api/teachers/{id}`	200 OK / 404	JSON object	yes
Create	`POST /api/teachers`	**201 Created** + `Location`	created object	no (two POSTs = two teachers)
Replace/update	`PUT /api/teachers/{id}`	200 OK	updated object	yes (same PUT twice = same result)
Delete	`DELETE /api/teachers/{id}`	**204 No Content**	empty	yes (second time → 404, but state is the same)

- URLs are **nouns** (resources), plural, lower-case. The HTTP method is the verb. No `/getTeacher` or `/deleteStudent`.

- The id lives in the path (`/teachers/5`), because it identifies the resource.
- `/api` prefix separates the API from anything else the server might serve.

src/main/java/com/university/ums/controller/TeacherController.java

```java
package com.university.ums.controller;

import com.university.ums.dto.TeacherRequest;
import com.university.ums.dto.TeacherResponse;
import com.university.ums.service.TeacherService;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;

/**
 * CRUD endpoints for teachers. Who may call them is decided in SecurityConfig
 * (/api/teachers/** rule), not here.
 */
@Tag(name = "Teachers", description = "Manage teachers (ADMIN or TEACHER)")
@RestController
@RequestMapping("/api/teachers")
public class TeacherController {

    private final TeacherService teacherService;

    public TeacherController(TeacherService teacherService) {
        this.teacherService = teacherService;
    }

    // GET /api/teachers -> 200 OK + list
    @GetMapping
    public List<TeacherResponse> getAll() {
        return teacherService.findAll();
    }

    // GET /api/teachers/{id} -> 200 OK, or 404 Not Found
    @GetMapping("/{id}")
    public TeacherResponse getById(@PathVariable Long id) {
        return teacherService.findById(id);
    }

    // POST /api/teachers -> 201 Created + Location header pointing at the new resource
    @PostMapping
    public ResponseEntity<TeacherResponse> create(@Valid @RequestBody TeacherRequest request) {
        TeacherResponse created = teacherService.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}")
                .buildAndExpand(created.id())
                .toUri();
        return ResponseEntity.created(location).body(created);
    }
```

```
61
62      // PUT /api/teachers/{id} -> 200 OK + updated resource
63      @PutMapping("/{id}")
64      public TeacherResponse update(@PathVariable Long id, @Valid @RequestBody TeacherRequest
request) {
65          return teacherService.update(id, request);
66      }
67
68      // DELETE /api/teachers/{id} -> 204 No Content
69      @DeleteMapping("/{id}")
70      @ResponseStatus(HttpStatus.NO_CONTENT)
71      public void delete(@PathVariable Long id) {
72          teacherService.delete(id);
73      }
74  }
```

Annotations and logic

- `@Tag(name = "Teachers", ...)`: documentation only; groups these endpoints under a "Teachers" heading in Swagger UI.
- `@RestController` = `@Controller` + `@ResponseBody`: every method's return value is written to the response as JSON (not a view/HTML page).
- `@RequestMapping("/api/teachers")`: prefix for every method in the class. `@GetMapping("/{id}")` then means `GET /api/teachers/{id}`.
- `@PathVariable Long id`: takes `5` from `/api/teachers/5` and converts it to `Long`. `/api/teachers/abc` cannot be converted → `MethodArgumentTypeMismatchException` → 400.
- `@RequestBody`: Jackson converts the JSON body to `TeacherRequest`. `@Valid`: run the validation annotations before the method body executes.
- Returning a plain object (e.g. `TeacherResponse`) gives status 200 by default.
- `create` returns `ResponseEntity` so it can set status and headers: `ServletUriComponentsBuilder.fromCurrentRequest()` starts from the current URL (`http://localhost:8080/api/teachers`), `.path("/{id}").buildAndExpand(created.id())` appends the new id. `ResponseEntity.created(location)` = status 201 + header `Location: http://localhost:8080/api/teachers/7`. The client now knows where the new resource lives.
- `delete` returns `void` with `@ResponseStatus(HttpStatus.NO_CONTENT)` → 204, empty body.
- The controller has **no** try/catch, no SQL, no security checks. Errors are thrown from the service and handled globally; access rules live in `SecurityConfig`. "Thin controller" = easy to read.

`AdminController` and `StudentController` are the same with `admin` / `student` names and DTOs.

AuthController – login and "who am I"

src/main/java/com/university/ums/controller/AuthController.java

```
1  package com.university.ums.controller;
2
3  import com.university.ums.dto.CurrentUserResponse;
4  import com.university.ums.dto.LoginRequest;
5  import com.university.ums.repository.UserRepository;
6  import io.swagger.v3.oas.annotations.tags.Tag;
7  import jakarta.servlet.http.HttpServletRequest;
8  import jakarta.servlet.http.HttpServletResponse;
9  import jakarta.validation.Valid;
10 import org.springframework.security.authentication.AuthenticationManager;
11 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
```

```java
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.context.SecurityContextRepository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

/**
 * Login and "who am I". Logout is handled by Spring Security's LogoutFilter
 * at POST /api/auth/logout (see SecurityConfig), so there is no method for it here.
 */
@Tag(name = "Authentication", description = "Log in, see who is logged in, log out")
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final SecurityContextRepository securityContextRepository;
    private final UserRepository userRepository;

    public AuthController(AuthenticationManager authenticationManager,
                          SecurityContextRepository securityContextRepository,
                          UserRepository userRepository) {
        this.authenticationManager = authenticationManager;
        this.securityContextRepository = securityContextRepository;
        this.userRepository = userRepository;
    }

    // POST /api/auth/login {"username": "...", "password": "..."} -> 200 + JSESSIONID cookie, or
401
    @PostMapping("/login")
    public CurrentUserResponse login(@Valid @RequestBody LoginRequest request,
                                     HttpServletRequest httpRequest,
                                     HttpServletResponse httpResponse) {
        // 1. Check username + password (loads user via CustomUserDetailsService, compares BCrypt
hash).
        //    Wrong credentials throw BadCredentialsException -> 401 in GlobalExceptionHandler.
        Authentication authentication = authenticationManager.authenticate(
                UsernamePasswordAuthenticationToken.unauthenticated(request.username(),
request.password()));

        // 2. Session fixation protection: never keep a session id that existed before login.
        if (httpRequest.getSession(false) != null) {
            httpRequest.changeSessionId();
        }

        // 3. Store the logged-in user in the HTTP session so later requests are authenticated
        //    just by sending the JSESSIONID cookie.
        SecurityContext context = SecurityContextHolder.createEmptyContext();
        context.setAuthentication(authentication);
        SecurityContextHolder.setContext(context);
        securityContextRepository.saveContext(context, httpRequest, httpResponse);

        return currentUser(authentication);
    }

    // GET /api/auth/me -> 200 with the current user's id, name, username and role
    @GetMapping("/me")
    public CurrentUserResponse me(Authentication authentication) {
        return currentUser(authentication);
    }

    private CurrentUserResponse currentUser(Authentication authentication) {
```

```
        return userRepository.findByUsername(authentication.getName())
                .map(CurrentUserResponse::from)
                .orElseThrow(() -> new IllegalStateException("Authenticated user no longer
exists"));
    }
}
```

Explained in detail in section 13.4 (login flow). In short: check the credentials with the `AuthenticationManager`, protect against session fixation, put the result into a `SecurityContext` and save that in the HTTP session. `/me` receives the current `Authentication` as a method parameter (Spring injects it) and looks up the user's details.

12. exception/ – consistent JSON errors

src/main/java/com/university/ums/exception/ApiError.java

```java
package com.university.ums.exception;

import com.fasterxml.jackson.annotation.JsonInclude;

import java.time.Instant;
import java.util.Map;

/** One JSON shape for every error the API returns. fieldErrors only appears on validation errors.
*/
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ApiError(
        Instant timestamp,
        int status,
        String error,
        String message,
        String path,
        Map<String, String> fieldErrors
) {
    public static ApiError of(int status, String error, String message, String path) {
        return new ApiError(Instant.now(), status, error, message, path, null);
    }
}
```

Every error from the API has the same shape, so a client always knows where to find the message. `@JsonInclude(NON_NULL)` omits `fieldErrors` unless it is a validation error.

```
{"timestamp":"2026-10-24T19:24:50.355Z","status":409,"error":"Conflict",
 "message":"Username 'rahim' is already taken","path":"/api/students"}

{"timestamp":"...","status":400,"error":"Bad Request","message":"Validation
failed","path":"/api/students",
 "fieldErrors":{"email":"must be a well-formed email address","batch":"must not be null", ...}}
```

src/main/java/com/university/ums/exception/ResourceNotFoundException.java

```java
package com.university.ums.exception;

/** Mapped to 404 Not Found by GlobalExceptionHandler. */
public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String resource, Long id) {
        super(resource + " with id " + id + " not found");
    }
}
```

`ConflictException` and `BadRequestException` look the same with a plain message. They extend `RuntimeException` (unchecked) so methods don't need `throws` clauses, and so `@Transactional` rolls back automatically when they are thrown.

src/main/java/com/university/ums/exception/GlobalExceptionHandler.java

```java
package com.university.ums.exception;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.security.core.AuthenticationException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

import java.time.Instant;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Turns exceptions thrown by controllers/services into JSON error responses
 * with the right HTTP status code, so controllers stay free of try/catch.
 */
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex, HttpServletRequest
request) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), request);
    }

    @ExceptionHandler(ConflictException.class)
    public ResponseEntity<ApiError> handleConflict(ConflictException ex, HttpServletRequest
request) {
        return build(HttpStatus.CONFLICT, ex.getMessage(), request);
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<ApiError> handleBadRequest(BadRequestException ex, HttpServletRequest
request) {
        return build(HttpStatus.BAD_REQUEST, ex.getMessage(), request);
    }

    // @Valid failed: collect every invalid field and its message
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex,
```

```
HttpServletRequest request) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            fieldErrors.putIfAbsent(error.getField(), error.getDefaultMessage());
        }
        ApiError body = new ApiError(Instant.now(), 400, "Bad Request", "Validation failed",
                request.getRequestURI(), fieldErrors);
        return ResponseEntity.badRequest().body(body);
    }

    // Body is not valid JSON, or a field has the wrong type (e.g. "batch": "abc")
    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ApiError> handleUnreadable(HttpMessageNotReadableException ex,
HttpServletRequest request) {
        return build(HttpStatus.BAD_REQUEST, "Malformed JSON request body", request);
    }

    // Path variable has the wrong type, e.g. /api/students/abc
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiError> handleTypeMismatch(MethodArgumentTypeMismatchException ex,
HttpServletRequest request) {
        return build(HttpStatus.BAD_REQUEST, "Invalid value '" + ex.getValue() + "' for parameter
'" + ex.getName() + "'", request);
    }

    // Safety net: the database rejected a duplicate the service checks did not catch (e.g. two
requests at once)
    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ApiError> handleIntegrity(DataIntegrityViolationException ex,
HttpServletRequest request) {
        return build(HttpStatus.CONFLICT, "Request conflicts with existing data (duplicate value)",
request);
    }

    // Wrong username/password on POST /api/auth/login
    @ExceptionHandler(AuthenticationException.class)
    public ResponseEntity<ApiError> handleAuthentication(AuthenticationException ex,
HttpServletRequest request) {
        return build(HttpStatus.UNAUTHORIZED, "Invalid username or password", request);
    }

    private ResponseEntity<ApiError> build(HttpStatus status, String message, HttpServletRequest
request) {
        ApiError body = ApiError.of(status.value(), status.getReasonPhrase(), message,
request.getRequestURI());
        return ResponseEntity.status(status).body(body);
    }
}
```

- `@RestControllerAdvice`: a global helper applied to all controllers. When a controller (or the service it calls) throws, Spring looks for an `@ExceptionHandler` method whose type matches, calls it, and sends its return value as the response.
- `handleValidation`: loops over every invalid field and puts `field → message` into a `LinkedHashMap` (keeps insertion order). `putIfAbsent` keeps only the first message per field.
- `handleAuthentication`: wrong login → 401 with a deliberately vague message. We do not say whether the username or the password was wrong; that would let an attacker discover which usernames exist (**user enumeration**).
- `build()`: small helper so every handler creates the same `ApiError`.

All status codes the API uses

Code	Meaning	When in this project	Produced by
200 OK	Success with a body	GET, PUT, login, logout, /me	controller default
201 Created	New resource created	POST to admins/teachers/students	`ResponseEntity.created`
204 No Content	Success, nothing to return	DELETE	`@ResponseStatus`
400 Bad Request	Client sent invalid data	validation failed, bad JSON, id not a number, no password on create	GlobalExceptionHandler
401 Unauthorized	Not authenticated (we don't know who you are)	no/expired session, wrong login	JsonSecurityHandlers / GlobalExceptionHandler
403 Forbidden	Authenticated, but not allowed	teacher → /api/students, student → /api/admins	JsonSecurityHandlers
404 Not Found	Resource doesn't exist	unknown id	ResourceNotFoundException
409 Conflict	Request conflicts with current state	duplicate username/email/studentId, deleting last admin	ConflictException, DataIntegrityViolation

13. Spring Security – login, logout, hashing, RBAC

13.1 Core vocabulary

Term	Meaning
Authentication	Proving **who you are** (username + password). Failure → 401.
Authorization	Deciding **what you may do** (your role vs the URL). Failure → 403.
Principal	The logged-in user (here: username).
Authority / Role	A permission string. A role is an authority with the prefix `ROLE_`: role TEACHER = authority `ROLE_TEACHER`.
`Authentication` object	Holds principal, (erased) credentials, authorities, and `isAuthenticated()`.
`SecurityContext`	Container holding the current `Authentication`.
`SecurityContextHolder`	Where the current request's context is kept (per thread) while the request is processed.
Session / `JSESSIONID`	Server-side storage per client, identified by a random id sent to the browser in a cookie.

13.2 The security filter chain

Spring Security is a chain of **servlet filters** that every request passes through *before* reaching any controller. With our configuration the important ones, in order, are:

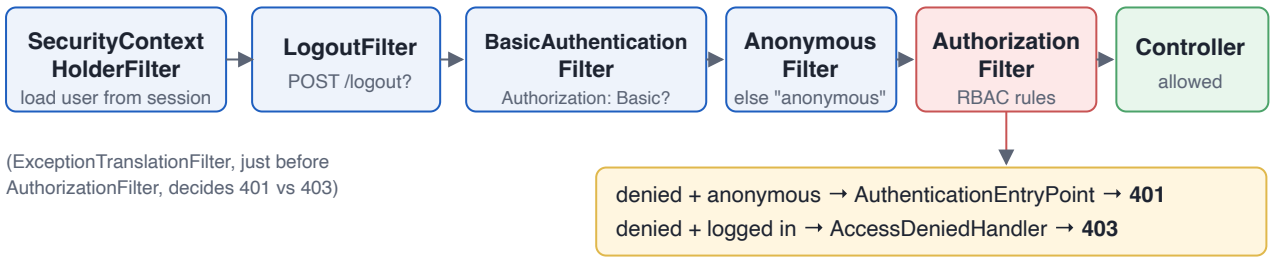


Figure 3 – Main filters, in order. Both 401 and 403 responses are written by our JsonSecurityHandlers.

> **How 401 vs 403 is decided.** If nobody logged in, `AnonymousAuthenticationFilter` puts an "anonymous" Authentication in the context. When `AuthorizationFilter` denies access it throws `AccessDeniedException`. `ExceptionTranslationFilter` catches it: if the user is anonymous, the problem is "we don't know who you are" → it calls the **AuthenticationEntryPoint** (401). If the user is a real logged-in user, the problem is "you may not" → it calls the **AccessDeniedHandler** (403).

13.3 SecurityConfig, line by line

`src/main/java/com/university/ums/security/SecurityConfig.java`

```
1 package com.university.ums.security;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
```

```java
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguratio
n;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
import org.springframework.security.web.context.SecurityContextRepository;
import org.springframework.security.web.servlet.util.matcher.PathPatternRequestMatcher;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http,
                                                   JsonSecurityHandlers jsonHandlers,
                                                   SecurityContextRepository
securityContextRepository) throws Exception {
        http
                // REST API used by Postman/curl/JS, not HTML forms. The session cookie is
SameSite=Lax
                // (application.yml), so browsers do not send it on cross-site POST/PUT/DELETE.
                .csrf(AbstractHttpConfigurer::disable)

                // RBAC: rules are checked top to bottom, first match wins
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()
                        .requestMatchers("/error").permitAll()
                        // Swagger UI page and the OpenAPI description it reads (docs only, no
data)
                        .requestMatchers("/swagger-ui.html", "/swagger-ui/**", "/v3/api-
docs/**").permitAll()
                        .requestMatchers("/api/admins/**").hasRole("ADMIN")
                        .requestMatchers("/api/teachers/**").hasAnyRole("ADMIN", "TEACHER")
                        .requestMatchers("/api/students/**").hasAnyRole("ADMIN", "STUDENT")
                        .anyRequest().authenticated())

                // Logged-in user is kept in the HTTP session (JSESSIONID cookie)
                .securityContext(context ->
context.securityContextRepository(securityContextRepository))

                // Also accept "Authorization: Basic ..." on every request (handy for Postman/curl)
                .httpBasic(basic -> basic.authenticationEntryPoint(jsonHandlers))

                // We use our own JSON login endpoint (AuthController), not the HTML login page
                .formLogin(AbstractHttpConfigurer::disable)

                .logout(logout -> logout
                        .logoutRequestMatcher(PathPatternRequestMatcher.withDefaults()
                                .matcher(HttpMethod.POST, "/api/auth/logout"))
                        .invalidateHttpSession(true)
                        .clearAuthentication(true)
                        .deleteCookies("JSESSIONID")
                        .logoutSuccessHandler(jsonHandlers))

                .exceptionHandling(ex -> ex
                        .authenticationEntryPoint(jsonHandlers)   // 401
                        .accessDeniedHandler(jsonHandlers));      // 403

        return http.build();
```

```
64      }
65
66      @Bean
67      public SecurityContextRepository securityContextRepository() {
68          return new HttpSessionSecurityContextRepository();
69      }
70
71      // BCrypt: slow, salted one-way hash. Same password -> different hash every time.
72      @Bean
73      public PasswordEncoder passwordEncoder() {
74          return new BCryptPasswordEncoder();
75      }
76
77      // Built by Spring Boot from CustomUserDetailsService + PasswordEncoder; used by AuthController
78      @Bean
79      public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration)
throws Exception {
80          return configuration.getAuthenticationManager();
81      }
82  }
```

- `@Configuration`: this class defines beans. `@EnableWebSecurity`: turn on our web security configuration.
- `securityFilterChain(HttpSecurity http, ...)`: `HttpSecurity` is a builder; each call configures one part of the chain. `http.build()` produces the `SecurityFilterChain` bean that Spring Boot installs. Spring passes in our `JsonSecurityHandlers` and `SecurityContextRepository` beans as parameters.
- `.csrf(AbstractHttpConfigurer::disable)`: turn off CSRF tokens. Discussed in 13.9.
- `.authorizeHttpRequests(...)`: the RBAC rules (13.8). Rules are tested **top to bottom; the first matching rule wins**.
  - `requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()`: anyone may try to log in (otherwise you would need to be logged in to log in).
  - `"/error"` permitAll: Spring Boot's internal error page; without it unexpected errors would turn into a confusing 401.
  - `/swagger-ui.html`, `/swagger-ui/**`, `/v3/api-docs/**` permitAll: the Swagger page and the API description must load before you log in (you log in *from* that page). They contain documentation only, no data; every data endpoint stays protected.
  - `/**` means "this path and everything under it".
  - `anyRequest().authenticated()`: everything else (e.g. `/api/auth/me`) just needs a logged-in user. Must be last (Spring refuses to start if a rule comes after it).
- `.securityContext(...)`: use `HttpSessionSecurityContextRepository`, i.e. load/store the logged-in user in the HTTP session.
- `.httpBasic(...)`: also accept `Authorization: Basic base64(username:password)` on any request. Convenient for Postman/curl/tests. Its entry point is set to our JSON handler; the default would send `WWW-Authenticate` and make browsers show a pop-up.
- `.formLogin(disable)`: no HTML login page; we have a JSON login endpoint.
- `.logout(...)`: `LogoutFilter` handles `POST /api/auth/logout` (only POST, built with `PathPatternRequestMatcher`, so a link or image can't log you out with a GET). It invalidates the session, clears the authentication, tells the browser to delete the `JSESSIONID` cookie, then calls our success handler (200 JSON).
- `.exceptionHandling(...)`: our JSON 401 and 403 writers.

- **Beans**: `SecurityContextRepository` (shared by the filter chain and `AuthController` so both use the same storage), `PasswordEncoder` = `BCryptPasswordEncoder`, and `AuthenticationManager`. Spring Boot sees our `UserDetailsService` and `PasswordEncoder` beans and automatically builds a `DaoAuthenticationProvider` from them; `configuration.getAuthenticationManager()` exposes that manager so we can inject it into `AuthController`.

13.4 Login flow, step by step

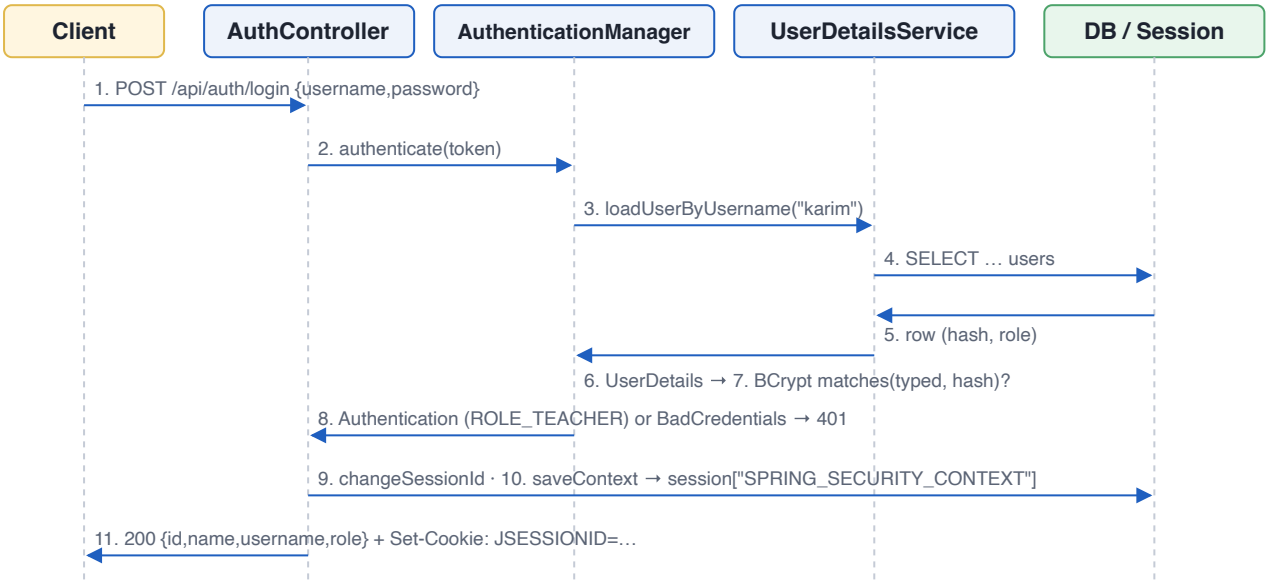


Figure 4 – Login sequence.

1. Client sends JSON `{"username":"karim","password":"secret123"}` to `POST /api/auth/login`. The rule `permitAll()` lets it through without being logged in. `@Valid` ensures both fields are not blank.
2. `UsernamePasswordAuthenticationToken.unauthenticated(username, password)` creates a "not yet verified" Authentication, passed to `authenticationManager.authenticate(...)`.
3. The manager delegates to `DaoAuthenticationProvider`, which calls our `CustomUserDetailsService.loadUserByUsername("karim")`.
4. That runs `userRepository.findByUsername` (after normalising), which searches all roles.
5. It returns a Spring Security `UserDetails` with username, the stored BCrypt hash and authority `ROLE_TEACHER`. If no user exists it throws `UsernameNotFoundException`.
6. The provider calls `passwordEncoder.matches("secret123", storedHash)`.
7. BCrypt re-hashes the typed password using the salt stored inside the hash and compares the results.
8. Match → a new, **authenticated** `Authentication` with the authorities. No match (or user not found) → `BadCredentialsException` (the provider hides "user not found" as "bad credentials" on purpose, and even hashes a dummy password so both cases take similar time — a protection against timing attacks). Our `GlobalExceptionHandler` turns it into 401.
9. **Session fixation protection**: if the client already had a session, `changeSessionId()` gives it a new id. Otherwise an attacker who planted a known session id in the victim's browser before login could use that same id after login.
10. An empty `SecurityContext` is created, the Authentication is put in it, it is set on `SecurityContextHolder` for this request, and `securityContextRepository.saveContext(...)` stores

it in the HTTP session under the attribute `SPRING_SECURITY_CONTEXT`. Since Spring Security 6 this explicit save is required; the framework no longer saves automatically.

11. Response 200 with the user's id/name/username/role. Tomcat adds `Set-Cookie: JSESSIONID=<random id>; HttpOnly; SameSite=Lax`.

src/main/java/com/university/ums/security/CustomUserDetailsService.java

```java
package com.university.ums.security;

import com.university.ums.model.AppUser;
import com.university.ums.repository.UserRepository;
import com.university.ums.service.UserAccountService;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

/**
 * Tells Spring Security how to find a user by username during login.
 * Spring then compares the typed password against the stored BCrypt hash itself.
 */
@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser user = userRepository.findByUsername(UserAccountService.normalize(username))
                .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));

        // roles("TEACHER") becomes the authority "ROLE_TEACHER", which hasRole("TEACHER") checks
        return User.withUsername(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRole().name())
                .build();
    }
}
```

`UserDetailsService` is the one interface Spring Security asks us to implement: "given a username, give me the stored password hash and authorities". Spring's builder `User.withUsername(...).password(...).roles("TEACHER")` automatically adds the `ROLE_` prefix.

13.5 Every request after login

1. The client (browser, Postman, curl with a cookie jar) sends `Cookie: JSESSIONID=...` automatically.
2. `SecurityContextHolderFilter` asks the `HttpSessionSecurityContextRepository` to find the session for that id and load `SPRING_SECURITY_CONTEXT`.
3. The context (with ROLE_TEACHER) is now available; `AuthorizationFilter` checks the rules against it.
4. No password is sent again, and the database is not queried for authentication.

The session expires after 30 minutes of inactivity (`server.servlet.session.timeout: 30m`).

13.6 HTTP Basic (alternative login per request)

Instead of logging in, a client may send `Authorization: Basic YWRtaW46YWRtaW4xMjM=` (Base64 of `admin:admin123`) with every request; `BasicAuthenticationFilter` authenticates it with the same AuthenticationManager. It does **not** create a session. Base64 is an **encoding, not encryption**; anyone who sees the header can decode it. That is why real deployments must use HTTPS. In Postman: Authorization tab → Basic Auth.

13.7 Logout

`POST /api/auth/logout` is handled entirely by Spring's `LogoutFilter` (that's why there is no controller method). It: (1) invalidates the HTTP session on the server, so the saved SecurityContext is gone; (2) clears the SecurityContextHolder; (3) sends `Set-Cookie: JSESSIONID=; Max-Age=0` to delete the cookie; (4) calls our `onLogoutSuccess` → 200 `{"message":"Logged out successfully"}`. Even if someone kept a copy of the old cookie, it is useless because the session it pointed to no longer exists → 401.

13.8 Role-based access control (RBAC)

URL pattern	Rule	ADMIN	TEACHER	STUDENT	Not logged in
`POST /api/auth/login`	`permitAll()`	✓	✓	✓	✓
`/api/admins/**`	`hasRole("ADMIN")`	✓	403	403	401
`/api/teachers/**`	`hasAnyRole("ADMIN","TEACHER")`	✓	✓	403	401
`/api/students/**`	`hasAnyRole("ADMIN","STUDENT")`	✓	403	✓	401
`/swagger-ui/**`, `/v3/api-docs/**`	`permitAll()`	✓	✓	✓	✓
anything else (`/api/auth/me`)	`authenticated()`	✓	✓	✓	401

- `hasRole("ADMIN")` checks for the authority `ROLE_ADMIN` (adds the prefix itself). `hasAuthority("ROLE_ADMIN")` would be the same thing written out.
- A rule on the URL covers **all** methods (GET/POST/PUT/DELETE), which is exactly "can manage" = full CRUD.
- The rules are enforced in a filter, **before** the controller runs: a forbidden request never reaches business code or the database.
- Alternative: **method security** — `@EnableMethodSecurity` + `@PreAuthorize("hasRole('ADMIN')")` on controller methods. URL rules were chosen because all rules are visible in one place; method security is better for fine-grained rules (see section 20).

13.9 Password hashing with BCrypt

- **Hashing vs encryption**: encryption can be reversed with a key; a hash is **one-way**. We never need the original password back; at login we hash what was typed and compare. If the database leaks, attackers only get hashes.

- **Salt**: BCrypt mixes a random 16-byte salt into every hash, so two users with the same password get different hashes, and pre-computed "rainbow tables" are useless.
- **Slow on purpose**: the **cost factor** (default 10) means 2^{10} = 1024 rounds of the expensive key setup. One login check takes tens of milliseconds — nothing for a user, but it makes guessing billions of passwords impractical. MD5/SHA-256 are designed to be fast, which is why they are unsuitable for passwords.
- **Format** of the stored value (real example from our MySQL): `$2a$10$dlWG43JPYk8ap647II8eb0…` → `2a` = BCrypt version, `10` = cost, next 22 characters = the salt, last 31 = the hash. The salt is stored inside the hash, so no separate column is needed. Total 60 characters.
- BCrypt only uses the first 72 bytes of the input, so the DTOs limit passwords to 72 characters.

13.10 CSRF, cookies and other security decisions

> **CSRF is disabled — why and is it safe?**
> CSRF (Cross-Site Request Forgery): while you are logged in, a malicious website makes *your browser* send a request to our API; the browser attaches your session cookie automatically. Spring's default defence is a CSRF token that each modifying request must include, which is designed for server-rendered HTML forms. For a JSON API used by Postman/curl/JS clients, that token makes testing much harder. We disabled it and instead set the session cookie to `SameSite=Lax`: modern browsers then **do not send the cookie on cross-site POST/PUT/DELETE requests**, which blocks the classic CSRF attack. Our only state-changing methods are POST/PUT/DELETE (no GET changes data), so Lax is enough. Clients using HTTP Basic are not affected by CSRF at all, since browsers don't attach that header to other sites' requests. If a browser front-end were added later, re-enabling CSRF with `CookieCsrfTokenRepository` would be the stronger option.

- `HttpOnly` cookie: JavaScript in the page cannot read `JSESSIONID`, so an XSS bug can't steal it.
- Same error for wrong username and wrong password (prevents user enumeration).
- Passwords never in responses or logs.
- Non-root user inside the Docker container; MySQL port not published to the host (section 16).

13.11 JsonSecurityHandlers – why not just use GlobalExceptionHandler?

src/main/java/com/university/ums/security/JsonSecurityHandlers.java

```
1  package com.university.ums.security;
2
3  import com.fasterxml.jackson.databind.ObjectMapper;
4  import com.university.ums.dto.MessageResponse;
5  import com.university.ums.exception.ApiError;
6  import jakarta.servlet.http.HttpServletRequest;
7  import jakarta.servlet.http.HttpServletResponse;
8  import org.springframework.http.HttpStatus;
9  import org.springframework.http.MediaType;
10 import org.springframework.security.access.AccessDeniedException;
11 import org.springframework.security.core.Authentication;
12 import org.springframework.security.core.AuthenticationException;
13 import org.springframework.security.web.AuthenticationEntryPoint;
14 import org.springframework.security.web.access.AccessDeniedHandler;
15 import org.springframework.security.web.authentication.logout.LogoutSuccessHandler;
16 import org.springframework.stereotype.Component;
```

```java

import java.io.IOException;

/**
 * Security errors happen in filters, before any controller runs, so @RestControllerAdvice
 * cannot see them. These handlers write the JSON responses instead of Spring's default
 * HTML login page / browser pop-up.
 */
@Component
public class JsonSecurityHandlers implements AuthenticationEntryPoint, AccessDeniedHandler,
LogoutSuccessHandler {

    private final ObjectMapper objectMapper;

    public JsonSecurityHandlers(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }

    // Not logged in (no session, no/invalid Basic credentials) -> 401 Unauthorized
    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
                         AuthenticationException authException) throws IOException {
        write(response, HttpStatus.UNAUTHORIZED, ApiError.of(401, "Unauthorized",
                "Authentication required. Log in at POST /api/auth/login",
request.getRequestURI()));
    }

    // Logged in, but the role is not allowed for this URL -> 403 Forbidden
    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       AccessDeniedException accessDeniedException) throws IOException {
        write(response, HttpStatus.FORBIDDEN, ApiError.of(403, "Forbidden",
                "Your role is not allowed to access this resource", request.getRequestURI()));
    }

    // POST /api/auth/logout finished (session invalidated, cookie deleted) -> 200 OK
    @Override
    public void onLogoutSuccess(HttpServletRequest request, HttpServletResponse response,
                                Authentication authentication) throws IOException {
        write(response, HttpStatus.OK, new MessageResponse("Logged out successfully"));
    }

    private void write(HttpServletResponse response, HttpStatus status, Object body) throws
IOException {
        response.setStatus(status.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        objectMapper.writeValue(response.getOutputStream(), body);
    }
}
```

401/403 decisions happen inside **filters**, which run *before* Spring MVC. `@RestControllerAdvice` only sees exceptions thrown from controllers, so it never sees them. Spring Security instead calls the `AuthenticationEntryPoint` (401) and `AccessDeniedHandler` (403) interfaces, and the `LogoutSuccessHandler` after logout. One class implements all three; each writes the status, the content type and the JSON (via Spring's shared `ObjectMapper`, so dates are formatted exactly like everywhere else). Without these, Spring would send an empty body / browser login pop-up / redirect to a login page.

14. The first admin: config/DataInitializer

src/main/java/com/university/ums/config/DataInitializer.java

```java
package com.university.ums.config;

import com.university.ums.model.Admin;
import com.university.ums.model.Student;
import com.university.ums.model.Teacher;
import com.university.ums.repository.AdminRepository;
import com.university.ums.repository.StudentRepository;
import com.university.ums.repository.TeacherRepository;
import com.university.ums.service.UserAccountService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

/**
 * Only admins can create admins, so an empty database would lock everyone out.
 * On startup, if there is no admin yet, create one from app.admin.* settings
 * (set APP_ADMIN_USERNAME / APP_ADMIN_PASSWORD / APP_ADMIN_EMAIL in Docker).
 * If app.demo-data is true, also add demo users for the lab demo.
 */
@Component
public class DataInitializer implements CommandLineRunner {

    private static final Logger log = LoggerFactory.getLogger(DataInitializer.class);

    private final AdminRepository adminRepository;
    private final TeacherRepository teacherRepository;
    private final StudentRepository studentRepository;
    private final UserAccountService accounts;

    @Value("${app.admin.username}")
    private String username;

    @Value("${app.admin.password}")
    private String password;

    @Value("${app.admin.email}")
    private String email;

    @Value("${app.demo-data:false}")
    private boolean demoData;

    public DataInitializer(AdminRepository adminRepository, TeacherRepository teacherRepository,
                           StudentRepository studentRepository, UserAccountService accounts) {
        this.adminRepository = adminRepository;
        this.teacherRepository = teacherRepository;
        this.studentRepository = studentRepository;
        this.accounts = accounts;
    }

    @Override
    @Transactional
    public void run(String... args) {
        // Runs only on an empty database, so restarts never create duplicates
        if (adminRepository.count() > 0) {
            return;
        }
        Admin admin = new Admin();
        accounts.applyCommonFields(admin, "System Administrator", email, null, username,
password);
        adminRepository.save(admin);
        log.info("Created initial admin account '{}'", admin.getUsername());

```

```
        if (demoData) {
            createDemoUsers();
        }
    }

    /**
     * Inserted in this order, so on a fresh database the ids are sequential:
     * admins 1–2, teachers 3–5, students 6–8 (all roles share the users table's id counter).
     */
    private void createDemoUsers() {
        admin("Nusrat Jahan", "admin2@university.edu", "admin2", "admin123");

        teacher("Dr. Abdul Karim", "karim.cse@university.edu", "teacher1", "Computer Science and
Engineering", "Professor");
        teacher("Farhana Akter", "farhana.eee@university.edu", "teacher2", "Electrical and
Electronic Engineering", "Associate Professor");
        teacher("Tanvir Hasan", "tanvir.me@university.edu", "teacher3", "Mechanical Engineering",
"Lecturer");

        student("Rahim Uddin", "rahim.2107001@university.edu", "student1", "2107001", "Computer
Science and Engineering", 2021);
        student("Sadia Islam", "sadia.2107002@university.edu", "student2", "2107002", "Computer
Science and Engineering", 2021);
        student("Arif Hossain", "arif.2203015@university.edu", "student3", "2203015", "Electrical
and Electronic Engineering", 2022);

        log.info("Created demo users: admin2, teacher1–3 (password teacher123), student1–3
(password student123)");
    }

    private void admin(String name, String email, String username, String rawPassword) {
        Admin admin = new Admin();
        accounts.applyCommonFields(admin, name, email, null, username, rawPassword);
        adminRepository.save(admin);
    }

    private void teacher(String name, String email, String username, String department, String
designation) {
        Teacher teacher = new Teacher();
        accounts.applyCommonFields(teacher, name, email, null, username, "teacher123");
        teacher.setDepartment(department);
        teacher.setDesignation(designation);
        teacherRepository.save(teacher);
    }

    private void student(String name, String email, String username, String studentId, String
department, int batch) {
        Student student = new Student();
        accounts.applyCommonFields(student, name, email, null, username, "student123");
        student.setStudentId(studentId);
        student.setDepartment(department);
        student.setBatch(batch);
        studentRepository.save(student);
    }
}
```

- **The bootstrap problem**: `/api/admins` requires ADMIN. A fresh, empty database has no admin, so nobody could ever log in or create anyone. Something must create the first admin outside the API.
- `CommandLineRunner`: Spring Boot calls `run()` once after the application has fully started (database ready, beans created).
- `@Value("${app.admin.username}")`: injects the value of that property from `application.yml` (or from the environment variable `APP_ADMIN_USERNAME`, which wins — section 15).

- **Idempotent**: if any admin already exists, it does nothing. Restarting the app never creates duplicates or resets the admin's password.
- It reuses `UserAccountService.applyCommonFields`, so the seeded password is BCrypt-hashed exactly like any other.
- `@Transactional`: the count and insert happen in one transaction.

Demo users (app.demo-data)

When `app.demo-data` is `true` (the default in `application.yml`), the same first-start run also inserts demo users through `createDemoUsers()`. The helper methods `admin()`, `teacher()` and `student()` build each entity, call `applyCommonFields` (so passwords are BCrypt-hashed like any other) and save it. Because all roles share the `users` table's AUTO_INCREMENT counter and the inserts happen in a fixed order, a fresh database gets sequential ids:

ID	Username	Password	Role	Name / details
1	admin	admin123	ADMIN	System Administrator
2	admin2	admin123	ADMIN	Nusrat Jahan
3	teacher1	teacher123	TEACHER	Dr. Abdul Karim — CSE, Professor
4	teacher2	teacher123	TEACHER	Farhana Akter — EEE, Associate Professor
5	teacher3	teacher123	TEACHER	Tanvir Hasan — ME, Lecturer
6	student1	student123	STUDENT	Rahim Uddin — 2107001, CSE, 2021
7	student2	student123	STUDENT	Sadia Islam — 2107002, CSE, 2021
8	student3	student123	STUDENT	Arif Hossain — 2203015, EEE, 2022

Ids are only 1–8 on a **fresh** database: MySQL never reuses ids, so after creating and deleting users the next id keeps counting up. Reset with `docker compose down -v`. Tests set `app.demo-data: false` because some tests expect exactly one admin. `@Value("${app.demo-data:false}")`: the `:false` is a default used if the property is missing.

> The default `admin / admin123` is for the lab demo only. For anything real, set `ADMIN_PASSWORD` in a `.env` file before the first start.

And the class that starts it all:

src/main/java/com/university/ums/UmsApplication.java

```java
package com.university.ums;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class UmsApplication {

    public static void main(String[] args) {
        SpringApplication.run(UmsApplication.class, args);
    }
}
```

`@SpringBootApplication` combines three annotations: `@Configuration` (this class may define beans), `@EnableAutoConfiguration` (configure libraries found on the classpath) and `@ComponentScan` (find our annotated classes in this package and below). `SpringApplication.run` creates the application context, starts Tomcat on port 8080, then runs `CommandLineRunner`s.

15. Configuration: application.yml

src/main/resources/application.yml

```yaml
spring:
  application:
    name: university-management-system
  datasource:
    # Defaults are for running on your laptop against a local MySQL.
    # In Docker, compose.yaml overrides them with SPRING_DATASOURCE_URL etc.
    url: jdbc:mysql://localhost:3306/university_db
    username: ums_user
    password: ums_password
  jpa:
    hibernate:
      # Create/alter tables from the @Entity classes on startup (fine for a lab project)
      ddl-auto: update
    open-in-view: false
    show-sql: false

server:
  port: 8080
  servlet:
    session:
      timeout: 30m
      cookie:
        http-only: true   # JavaScript cannot read the session cookie
        same-site: lax    # cookie not sent on cross-site POST/PUT/DELETE (CSRF protection)

# First admin, created on startup only if no admin exists (see DataInitializer).
# Override with environment variables APP_ADMIN_USERNAME, APP_ADMIN_PASSWORD, APP_ADMIN_EMAIL.
app:
  admin:
    username: admin
    password: admin123
    email: admin@university.edu
  # Also create demo users on a fresh database (admin2, teacher1-3, student1-3). Set
APP_DEMO_DATA=false to skip.
  demo-data: true

# Swagger UI (http://localhost:8080/swagger-ui.html)
springdoc:
  swagger-ui:
    tags-sorter: alpha            # Admins, Authentication, Students, Teachers
    operations-sorter: method     # GET, POST, PUT, DELETE order inside each group
    try-it-out-enabled: true      # request forms are editable without an extra click
    persist-authorization: true   # keep the Authorize credentials after a page reload
```

- `spring.datasource.*`: JDBC URL, username and password for MySQL. Defaults point to `localhost` for running without Docker.
- `ddl-auto: update`: at startup Hibernate compares entities with the tables and creates missing tables/columns. It never deletes data. Good for a lab; real projects use migration tools such as Flyway (section 20).

- `open-in-view: false`: by default Spring keeps the database session open until the JSON is written, which hides lazy-loading problems and holds connections longer. We turn it off; all mapping to DTOs happens inside service transactions.
- `session.timeout`, `cookie.http-only`, `cookie.same-site`: see 13.5 and 13.10.
- `app.admin.*`: our own properties, read by DataInitializer.
- `springdoc.swagger-ui.*`: sort groups alphabetically and operations by method, open request forms ready to edit, and keep the Authorize credentials after a page reload. `persist-authorization` stores those credentials in the browser's localStorage, which is a demo convenience; turn it off on a shared computer.

How Docker overrides these values (relaxed binding)

Spring Boot reads configuration from many places; **environment variables have higher priority than application.yml**. It also maps names automatically: upper-case, dots → underscores.

Property in application.yml	Environment variable that overrides it
`spring.datasource.url`	`SPRING_DATASOURCE_URL`
`spring.datasource.password`	`SPRING_DATASOURCE_PASSWORD`
`app.admin.password`	`APP_ADMIN_PASSWORD`

So the same jar runs on a laptop (localhost) and in Docker (host `db`) without code changes. The test configuration (`src/test/resources/application.yml`) replaces the datasource with H2 in MySQL compatibility mode and uses `create-drop` (fresh tables every test run).

16. Docker and Docker Compose

16.1 Vocabulary

Term	Meaning
Image	A read-only template: an operating system layer + Java + our jar. Built from a Dockerfile.
Container	A running instance of an image (an isolated process with its own filesystem and network).
Layer	Each Dockerfile instruction creates a layer. Unchanged layers are reused from cache → fast rebuilds.
Volume	Storage managed by Docker that survives when a container is deleted (database files).
Network	Compose puts all services on one private network where each service name is a hostname.
Compose	Tool that starts several containers together from one YAML file.

16.2 Dockerfile, line by line

Dockerfile

```dockerfile
# ---------- Stage 1: build the jar with Maven + JDK ----------
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

# Copy only pom.xml first and download dependencies. Docker caches this layer,
# so later builds skip the download unless pom.xml changes.
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

# Now copy the source code and build the executable jar (tests run separately)
COPY src ./src
RUN mvn -B -q package -DskipTests

# ---------- Stage 2: small runtime image with only the JRE + jar ----------
FROM eclipse-temurin:21-jre
WORKDIR /app

# Do not run the application as root inside the container
RUN groupadd --system spring && useradd --system --gid spring spring

COPY --from=build /app/target/university-management-system-1.0.0.jar app.jar

USER spring
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

- **Multi-stage build**: there are two `FROM` lines. Stage 1 (`build`) has Maven + the full JDK to compile. Stage 2 starts again from a small JRE-only image and copies **just the jar** from stage 1 (`COPY --from=build`). The final image contains no Maven, no source code, no compiler and no downloaded build cache: smaller, faster to ship, less to attack.
- `WORKDIR /app`: working directory inside the image (created if missing).
- **Layer caching trick**: `COPY pom.xml` + `mvn dependency:go-offline` come *before* `COPY src`. Dependencies change rarely, source code often. When only Java code changes, Docker reuses the cached dependency layer and skips re-downloading hundreds of megabytes.

- `mvn -B -q package -DskipTests`: `-B` batch mode (no colours/prompts), `-q` quiet, build the jar to `target/`. Tests are skipped here because they are run separately with `./mvnw test`; this keeps image builds fast. (Trade-off: you could run them in the build instead.)
- `groupadd/useradd` + `USER spring`: by default containers run as root. Running as an unprivileged user limits the damage if the app were ever compromised.
- `EXPOSE 8080`: documents the port the app listens on (does not publish it by itself; compose's `ports` does).
- `ENTRYPOINT ["java", "-jar", "/app/app.jar"]`: the command run when the container starts. The JSON "exec form" makes Java the main process (PID 1), so `docker stop` signals reach it and Spring shuts down cleanly.

16.3 compose.yaml, line by line

compose.yaml

```yaml
# Start everything:  docker compose up --build
# Stop:              docker compose down        (add -v to also delete the database volume)
# Values like ${DB_PASSWORD:-ums_password} come from a .env file if present, else the default after ":-".

services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: ${DB_NAME:-university_db}
      MYSQL_USER: ${DB_USER:-ums_user}
      MYSQL_PASSWORD: ${DB_PASSWORD:-ums_password}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:-root_password}
    volumes:
      - db-data:/var/lib/mysql          # data survives container restarts
    healthcheck:
      # -h 127.0.0.1 forces TCP, which only answers once MySQL has finished its first-time setup
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u$$MYSQL_USER -p$$MYSQL_PASSWORD --silent"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 20s
    restart: unless-stopped
    # No "ports:" on purpose: only the app container can reach MySQL (over the compose network).

  app:
    build: .
    depends_on:
      db:
        condition: service_healthy      # wait until MySQL accepts connections
    environment:
      # "db" is the service name above; compose's internal DNS resolves it to the MySQL container
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/${DB_NAME:-university_db}
      SPRING_DATASOURCE_USERNAME: ${DB_USER:-ums_user}
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD:-ums_password}
      APP_ADMIN_USERNAME: ${ADMIN_USERNAME:-admin}
      APP_ADMIN_PASSWORD: ${ADMIN_PASSWORD:-admin123}
      APP_ADMIN_EMAIL: ${ADMIN_EMAIL:-admin@university.edu}
    ports:
      - "8080:8080"                      # host:container
    restart: unless-stopped

volumes:
  db-data:
```

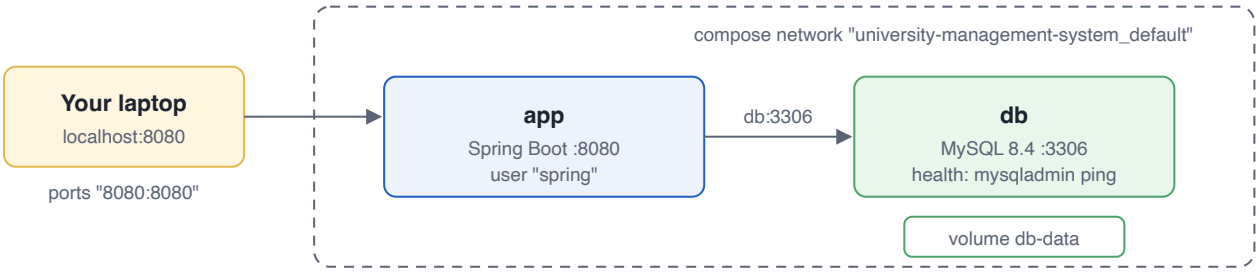


Figure 5 – Containers started by `docker compose up`. Only the app's port is published to the laptop.

- `services:` two containers, `db` and `app`.

- **db**: uses the official `mysql:8.4` image. The `MYSQL_*` variables tell the image, on first start, to create the database, an application user with that password, and the root password.
- `${DB_PASSWORD:-ums_password}`: compose **variable substitution** — take `DB_PASSWORD` from the shell or the `.env` file; if it is not set, use `ums_password`. Real passwords therefore live in `.env` (git-ignored), not in the YAML.
- `volumes: - db-data:/var/lib/mysql`: MySQL stores its files in a named volume. `docker compose down` keeps the data; `docker compose down -v` deletes it.
- **healthcheck**: every 5 s compose runs `mysqladmin ping` inside the container. The container is "healthy" only when it answers. Two details: (1) `$$` escapes the dollar sign so compose does not substitute it; the shell inside the container expands `$MYSQL_USER` from the container's own environment. (2) `-h 127.0.0.1` forces a TCP connection: on the very first start the MySQL image runs a temporary server *without networking* while it creates the database; a ping through the local socket would succeed too early, the TCP ping only succeeds once the real server is up.
- No `ports:` for db: MySQL is reachable only from other containers in the compose network, not from the laptop or the internet. (Add `"3306:3306"` if you want to connect a DB tool or run the app outside Docker.)
- **app**: `build: .` builds the image from the Dockerfile in this folder.
- `depends_on: db: condition: service_healthy`: compose starts `app` only after `db`'s health check passes. Plain `depends_on` would only wait for the container to *start*, and Spring would crash trying to connect to a MySQL that isn't ready yet.
- `SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/...`: `db` is the service name; compose's built-in DNS resolves it to the db container's IP. Inside a container, `localhost` would mean the app container itself.
- `APP_ADMIN_*`: the first admin's credentials (section 14).
- `ports: "8080:8080"`: *host port : container port*. Makes the API reachable at `http://localhost:8080`.
- `restart: unless-stopped`: Docker restarts the container if it crashes, unless you stopped it yourself.
- The bottom `volumes: db-data:` declares the named volume.

16.4 Helper files

.env.example

```
# Copy to .env and change the values. docker compose reads .env automatically.
DB_NAME=university_db
DB_USER=ums_user
DB_PASSWORD=change_me
DB_ROOT_PASSWORD=change_me_too
ADMIN_USERNAME=admin
ADMIN_PASSWORD=change_me_admin
ADMIN_EMAIL=admin@university.edu
```

.dockerignore

```
target/
.git/
.idea/
.vscode/
*.iml
.env
docs/
```

`.dockerignore` keeps the build context small and stops secrets (`.env`) and old build output (`target/`) from being sent into the image build.

16.5 Commands you will use

Command	What it does
`docker compose up --build`	Build the app image and start db + app (foreground; add `-d` for background)
`docker compose ps`	List the containers and their health
`docker compose logs -f app`	Follow the Spring Boot log
`docker compose exec db mysql -uums_user -pums_password university_db`	Open a MySQL prompt inside the db container (show the tables / hashes to the teacher)
`docker compose down`	Stop and remove containers (data kept in the volume)
`docker compose down -v`	Also delete the database volume (fresh start)

17. Automated tests

Run with `./mvnw test` (16 tests). They need no MySQL and no Docker: the test configuration uses an in-memory H2 database.

src/test/java/com/university/ums/UniversityApiTests.java

```
  1 package com.university.ums;
  2
  3 import com.university.ums.model.Teacher;
  4 import com.university.ums.repository.TeacherRepository;
  5 import org.junit.jupiter.api.Test;
  6 import org.springframework.beans.factory.annotation.Autowired;
  7 import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
  8 import org.springframework.boot.test.context.SpringBootTest;
  9 import org.springframework.http.MediaType;
 10 import org.springframework.mock.web.MockHttpSession;
 11 import org.springframework.test.web.servlet.MockMvc;
 12 import org.springframework.test.web.servlet.MvcResult;
 13 import org.springframework.transaction.annotation.Transactional;
 14
 15 import static org.assertj.core.api.Assertions.assertThat;
 16 import static org.hamcrest.Matchers.containsString;
 17 import static
org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.httpBasic;
 18 import static
org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
 19 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
 20 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
 21 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
 22 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.put;
 23 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
 24 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
 25 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
 26
 27 /**
 28  * End-to-end tests through the real security filter chain, controllers, services and an H2
database.
 29  * Each test runs in a transaction that is rolled back, so tests do not affect each other.
```

```
 */
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class UniversityApiTests {

    @Autowired
    MockMvc mvc;

    @Autowired
    TeacherRepository teacherRepository;

    private static final String TEACHER_JSON = """
            {"name":"Dr. Karim","email":"karim@uni.edu","username":"karim","password":"secret123",
             "department":"CSE","designation":"Professor"}""";

    private static final String STUDENT_JSON = """
            {"name":"Rahim","email":"rahim@uni.edu","username":"rahim","password":"secret123",
             "studentId":"2007001","department":"CSE","batch":2020}""";

    private static final String ADMIN_JSON = """
            {"name":"Second
Admin","email":"admin2@uni.edu","username":"admin2","password":"secret123"}""";

    private MockHttpSession login(String username, String password) throws Exception {
        MvcResult result = mvc.perform(post("/api/auth/login")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{\"username\":\"" + username + "\",\"password\":\"" + password +
"\"}"))
                .andExpect(status().isOk())
                .andReturn();
        return (MockHttpSession) result.getRequest().getSession(false);
    }

    // ---------- Authentication ----------

    @Test
    void requestWithoutLoginIsRejectedWith401Json() throws Exception {
        mvc.perform(get("/api/students"))
                .andExpect(status().isUnauthorized())
                .andExpect(jsonPath("$.status").value(401));
    }

    @Test
    void loginWithSeededAdminCreatesSessionAndMeWorks() throws Exception {
        MockHttpSession session = login("admin", "admin123");
        mvc.perform(get("/api/auth/me").session(session))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.username").value("admin"))
                .andExpect(jsonPath("$.role").value("ADMIN"));
    }

    @Test
    void loginWithWrongPasswordReturns401() throws Exception {
        mvc.perform(post("/api/auth/login")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{\"username\":\"admin\",\"password\":\"wrong\"}"))
                .andExpect(status().isUnauthorized())
                .andExpect(jsonPath("$.message").value("Invalid username or password"));
    }

    @Test
    void logoutInvalidatesSession() throws Exception {
        MockHttpSession session = login("admin", "admin123");
        mvc.perform(post("/api/auth/logout").session(session))
                .andExpect(status().isOk())
```

```
                .andExpect(jsonPath("$.message").value("Logged out successfully"));
        assertThat(session.isInvalid()).isTrue();
        mvc.perform(get("/api/auth/me").session(new MockHttpSession()))
                .andExpect(status().isUnauthorized());
    }

    @Test
    void httpBasicAuthenticationAlsoWorks() throws Exception {
        mvc.perform(get("/api/admins").with(httpBasic("admin", "admin123")))
                .andExpect(status().isOk());
    }

    // ---------- Swagger UI ----------

    @Test
    void swaggerUiAndApiDocsArePublic() throws Exception {
        mvc.perform(get("/swagger-ui/index.html"))
                .andExpect(status().isOk());
        mvc.perform(get("/v3/api-docs"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.paths['/api/teachers/{id}']").exists())
                .andExpect(jsonPath("$.paths['/api/auth/logout'].post").exists())

.andExpect(jsonPath("$.components.securitySchemes.basicAuth.scheme").value("basic"));
    }

    // ---------- Password hashing ----------

    @Test
    void passwordIsStoredAsBcryptHashAndNeverReturned() throws Exception {
        mvc.perform(post("/api/teachers").with(httpBasic("admin", "admin123"))
                        .contentType(MediaType.APPLICATION_JSON).content(TEACHER_JSON))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.password").doesNotExist());

        Teacher saved = teacherRepository.findAll().get(0);
        assertThat(saved.getPassword()).startsWith("$2a$").isNotEqualTo("secret123");

        // and the new teacher can log in with the original password
        login("karim", "secret123");
    }

    // ---------- CRUD + status codes ----------

    @Test
    void fullCrudLifecycleForStudent() throws Exception {
        MockHttpSession admin = login("admin", "admin123");

        MvcResult created = mvc.perform(post("/api/students").session(admin)
                        .contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON))
                .andExpect(status().isCreated())
                .andExpect(header().string("Location", containsString("/api/students/")))
                .andExpect(jsonPath("$.role").value("STUDENT"))
                .andReturn();
        String location = created.getResponse().getHeader("Location");

        mvc.perform(get(location).session(admin))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.studentId").value("2007001"));

        mvc.perform(put(location).session(admin).contentType(MediaType.APPLICATION_JSON)
                        .content(STUDENT_JSON.replace("\"batch\":2020", "\"batch\":2021")))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.batch").value(2021));

        mvc.perform(delete(location).session(admin))
```

```
                .andExpect(status().isNoContent());

        mvc.perform(get(location).session(admin))
                .andExpect(status().isNotFound());
    }

    @Test
    void invalidBodyReturns400WithFieldErrors() throws Exception {
        mvc.perform(post("/api/students").with(user("a").roles("ADMIN"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("{\"name\":\"\",\"email\":\"not-an-email\",\"username\":\"x
y\"}"))
                .andExpect(status().isBadRequest())
                .andExpect(jsonPath("$.fieldErrors.email").exists())
                .andExpect(jsonPath("$.fieldErrors.name").exists())
                .andExpect(jsonPath("$.fieldErrors.batch").exists());
    }

    @Test
    void createWithoutPasswordReturns400() throws Exception {
        mvc.perform(post("/api/teachers").with(user("a").roles("ADMIN"))
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(TEACHER_JSON.replace("\"password\":\"secret123\",", "")))
                .andExpect(status().isBadRequest());
    }

    @Test
    void duplicateUsernameAcrossRolesReturns409() throws Exception {
        String studentNamedAdmin = STUDENT_JSON.replace("\"username\":\"rahim\"",
"\"username\":\"Admin\"");
        mvc.perform(post("/api/students").with(user("a").roles("ADMIN"))
                        .contentType(MediaType.APPLICATION_JSON).content(studentNamedAdmin))
                .andExpect(status().isConflict());
    }

    @Test
    void unknownIdReturns404AndBadIdReturns400() throws Exception {
        mvc.perform(get("/api/teachers/9999").with(user("a").roles("ADMIN")))
                .andExpect(status().isNotFound());
        mvc.perform(get("/api/teachers/abc").with(user("a").roles("ADMIN")))
                .andExpect(status().isBadRequest());
    }

    @Test
    void lastAdminCannotBeDeleted() throws Exception {
        MockHttpSession admin = login("admin", "admin123");
        MvcResult me = mvc.perform(get("/api/auth/me").session(admin)).andReturn();
        String id = com.jayway.jsonpath.JsonPath.read(me.getResponse().getContentAsString(),
"$.id").toString();

        mvc.perform(delete("/api/admins/" + id).session(admin))
                .andExpect(status().isConflict());
    }

    // ---------- Role-based access control ----------

    @Test
    void adminCanManageAllThreeResources() throws Exception {
        MockHttpSession admin = login("admin", "admin123");

mvc.perform(post("/api/admins").session(admin).contentType(MediaType.APPLICATION_JSON).content(ADMIN_J
SON))
                .andExpect(status().isCreated());

mvc.perform(post("/api/teachers").session(admin).contentType(MediaType.APPLICATION_JSON).content(TEACH
ER_JSON))
```

```
                .andExpect(status().isCreated());

mvc.perform(post("/api/students").session(admin).contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON))
                .andExpect(status().isCreated());
    }

    @Test
    void teacherCanManageTeachersOnly() throws Exception {
        mvc.perform(post("/api/teachers").with(httpBasic("admin", "admin123"))

.contentType(MediaType.APPLICATION_JSON).content(TEACHER_JSON)).andExpect(status().isCreated());
        MockHttpSession teacher = login("karim", "secret123");

        mvc.perform(get("/api/teachers").session(teacher)).andExpect(status().isOk());
        mvc.perform(post("/api/teachers").session(teacher).contentType(MediaType.APPLICATION_JSON)
                        .content(TEACHER_JSON.replace("karim", "salma")))
                .andExpect(status().isCreated());

        mvc.perform(get("/api/students").session(teacher)).andExpect(status().isForbidden());

mvc.perform(post("/api/students").session(teacher).contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON))
                .andExpect(status().isForbidden());
        mvc.perform(get("/api/admins").session(teacher))
                .andExpect(status().isForbidden())
                .andExpect(jsonPath("$.status").value(403));
    }

    @Test
    void studentCanManageStudentsOnly() throws Exception {
        mvc.perform(post("/api/students").with(httpBasic("admin", "admin123"))

.contentType(MediaType.APPLICATION_JSON).content(STUDENT_JSON)).andExpect(status().isCreated());
        MockHttpSession student = login("rahim", "secret123");

        mvc.perform(get("/api/students").session(student)).andExpect(status().isOk());
        mvc.perform(post("/api/students").session(student).contentType(MediaType.APPLICATION_JSON)
                        .content(STUDENT_JSON.replace("rahim", "karima").replace("2007001", "2007002")))
                .andExpect(status().isCreated());

        mvc.perform(get("/api/teachers").session(student)).andExpect(status().isForbidden());
        mvc.perform(delete("/api/admins/1").session(student)).andExpect(status().isForbidden());
    }
}
```

How the tests work

- `@SpringBootTest`: starts the **whole** application (all beans, security, JPA) — these are integration tests, not unit tests.
- `@AutoConfigureMockMvc` + `MockMvc`: sends simulated HTTP requests through the real filter chain and controllers without opening a network port. `.andExpect(status().isCreated())` and `jsonPath("$.role")` check the response.
- `@Transactional` on the class: each test runs in a transaction that is **rolled back** at the end, so data created in one test never affects another. The seeded admin (created at startup) is always there.
- Three ways to be logged in: `login(...)` does the real `POST /api/auth/login` and reuses the returned session (tests the real login); `httpBasic("admin","admin123")` sends a Basic header;

`user("a").roles("ADMIN")` fakes an already-authenticated user (quick way to test rules that are not about login).

- Text blocks (`"""..."""`) hold JSON bodies; `.replace(...)` creates variants (another username, another batch).

Test	Proves
requestWithoutLoginIsRejectedWith401Json	Unauthenticated → 401 JSON
loginWithSeededAdmin…	Seeded admin exists, login creates a session, /me works
loginWithWrongPasswordReturns401	Wrong password → 401 with generic message
logoutInvalidatesSession	Logout → 200, session invalid, afterwards 401
httpBasicAuthenticationAlsoWorks	Basic auth accepted
swaggerUiAndApiDocsArePublic	Swagger page and OpenAPI JSON load without login; the JSON contains our endpoints, the logout path and the Basic security scheme
passwordIsStoredAsBcryptHash…	No password in JSON; DB holds `$2a$…` hash; the user can log in with the original password
fullCrudLifecycleForStudent	201 + Location, 200 GET, 200 PUT, 204 DELETE, then 404
invalidBodyReturns400WithFieldErrors / createWithoutPasswordReturns400	Validation and password-on-create rule
duplicateUsernameAcrossRolesReturns409	Student cannot take "Admin" (case-insensitive, cross-role)
unknownIdReturns404AndBadIdReturns400	404 and 400 for ids
lastAdminCannotBeDeleted	409 lock-out protection
adminCan… / teacherCan… / studentCan…	The full RBAC matrix from the assignment

Result: `Tests run: 16, Failures: 0, Errors: 0`.

18. Demonstrating it in the lab

1. Start Docker Desktop. In the project folder: `docker compose up --build`. Wait for the log line `Started UmsApplication` and `Created initial admin account 'admin'` (first start only).
2. In a second terminal: `./demo.sh`. It prints every request with its status code and JSON, in this order: 401 without login → admin login → create teacher and student (201) → duplicate (409) and invalid body (400) → teacher login: teachers 200, students/admins 403 → student login: students 200, teachers 403 → update (200), delete (204), get deleted (404) → logout (200) then 401.
3. Log in as any demo user (section 14 table), e.g. `teacher1 / teacher123`, `student1 / student123`.
4. Show the hashes: `docker compose exec db mysql -uums_user -pums_password university_db -e "SELECT id, username, password FROM users"`.
5. Show it in the browser: `http://localhost:8080/swagger-ui.html` (18.2).
6. Show the tests: `./mvnw test`.

18.1 The demo script

demo.sh

```
#!/usr/bin/env bash
# Walks through login, CRUD, RBAC and logout against a running app.
# Usage: ./demo.sh            (app must be running: docker compose up --build)
#        BASE=http://host:8080 ./demo.sh
set -u
BASE=${BASE:-http://localhost:8080}
JAR=$(mktemp -d)
trap 'rm -rf "$JAR"' EXIT

step() { printf '\n\033[1;34m== %s\033[0m\n' "$1"; }
# call <cookie-jar-name> <METHOD> <path> [json-body]: prints status code and body
call() {
  local jar="$JAR/$1" method=$2 path=$3 body=${4:-}
  if [ -n "$body" ]; then
    curl -s -o "$JAR/out" -w '%{http_code}' -b "$jar" -c "$jar" -X "$method" "$BASE$path" \
         -H 'Content-Type: application/json' -d "$body"
  else
    curl -s -o "$JAR/out" -w '%{http_code}' -b "$jar" -c "$jar" -X "$method" "$BASE$path"
  fi
  printf '  %s %s\n' "$method" "$path"; cat "$JAR/out"; echo
}

step "1. Not logged in -> 401"
call anon GET /api/students

step "2. Admin logs in (seeded account) -> 200 + JSESSIONID cookie"
call admin POST /api/auth/login '{"username":"admin","password":"admin123"}'

step "3. Admin creates a teacher and a student -> 201 Created"
call admin POST /api/teachers '{"name":"Dr.
Karim","email":"karim@uni.edu","username":"karim","password":"secret123","department":"CSE","designati
on":"Professor"}'
call admin POST /api/students
'{"name":"Rahim","email":"rahim@uni.edu","username":"rahim","password":"secret123","studentId":"200700
1","department":"CSE","batch":2020}'

step "4. Duplicate username -> 409, invalid body -> 400"
call admin POST /api/students
'{"name":"X","email":"x@uni.edu","username":"rahim","password":"secret123","studentId":"2007999","depa
rtment":"CSE","batch":2020}'
call admin POST /api/students '{"name":"","email":"bad","username":"a b"}'

step "5. Teacher logs in: teachers OK, students/admins -> 403"
call teacher POST /api/auth/login '{"username":"karim","password":"secret123"}'
call teacher GET /api/teachers
call teacher GET /api/students
call teacher GET /api/admins

step "6. Student logs in: students OK, teachers -> 403"
call student POST /api/auth/login '{"username":"rahim","password":"secret123"}'
call student GET /api/students
call student GET /api/teachers

step "7. Admin updates and deletes the student -> 200, 204, then 404"
ID=$(curl -s -b "$JAR/admin" "$BASE/api/students" | sed -n 's/.*"id":\([0-
9]*\).*"username":"rahim".*/\1/p')
call admin PUT "/api/students/$ID" '{"name":"Rahim
Uddin","email":"rahim@uni.edu","username":"rahim","studentId":"2007001","department":"CSE","batch":202
1}'
call admin DELETE "/api/students/$ID"
call admin GET "/api/students/$ID"
```

```
53 TID=$(curl -s -b "$JAR/admin" "$BASE/api/teachers" | sed -n 's/.*"id":\([0-
9]*\).*"username":"karim".*/\1/p')
54 call admin DELETE "/api/teachers/$TID"
55
56 step "8. Logout -> 200, then the same client is no longer logged in -> 401"
57 call admin POST /api/auth/logout
58 call admin GET /api/auth/me
```

`curl -b jar -c jar` reads/writes a "cookie jar" file, which is how curl remembers `JSESSIONID` between requests (like a browser). Each role gets its own jar so the three logins don't overwrite each other. `-w '%{http_code}'` prints the status code.

18.2 Swagger UI – testing from the browser

Swagger UI is a web page generated from our code. **springdoc** scans every `@RestController` at startup and builds an **OpenAPI** description (a standard JSON format describing every URL, method, request body, response and security scheme), served at `/v3/api-docs`. The Swagger UI page at `/swagger-ui.html` reads that JSON and draws a form for each endpoint with an **Execute** button that sends the real request from your browser.

How to use it

1. Open `http://localhost:8080/swagger-ui.html`. Endpoints are grouped: Admins, Authentication, Students, Teachers.
2. **Log in, way 1 (session):** Authentication → `POST /api/auth/login` → Execute (the body is pre-filled with `admin` / `admin123`; change it to log in as a teacher or student). The response is 200 and the browser stores the `JSESSIONID` cookie, which it sends with every later request from this page.
3. **Log in, way 2 (HTTP Basic):** click **Authorize** (top right), enter username and password, Authorize → Close. Swagger UI now adds an `Authorization: Basic …` header to every request (visible in the generated curl command).
4. Open any endpoint, edit the JSON or the id, click **Execute**. You see the status code, response body and headers (e.g. `Location` after a POST).
5. To show RBAC: log in as a teacher and execute `GET /api/students` → 403.
6. Log out: Authentication → `POST /api/auth/logout` → Execute, then any protected endpoint → 401. If you used Authorize, also open Authorize → Logout, otherwise the Basic header keeps you logged in.

> **Use one login method at a time.** When a request carries both a session cookie and a Basic header for a *different* user, `BasicAuthenticationFilter` re-authenticates with the Basic credentials, so the Authorize user wins. Clear Authorize before testing session logins as other roles.

Verified on the Docker stack: session login 200 → `GET /api/teachers` 200 → logout 200 → `GET /api/teachers` 401 → Authorize as admin → `GET /api/admins` 200.

OpenApiConfig – the code behind it

`src/main/java/com/university/ums/config/OpenApiConfig.java`

```java
package com.university.ums.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.Operation;
import io.swagger.v3.oas.models.PathItem;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.responses.ApiResponse;
import io.swagger.v3.oas.models.responses.ApiResponses;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.List;

/**
 * Describes the API for Swagger UI (http://localhost:8080/swagger-ui.html).
 * springdoc scans the controllers for endpoints; this bean adds the title, the
 * "Authorize" button (HTTP Basic) and the logout endpoint, which is a Spring Security
 * filter rather than a controller method, so the scanner cannot find it.
 */
@Configuration
public class OpenApiConfig {

    private static final String BASIC_AUTH = "basicAuth";

    @Bean
    public OpenAPI universityOpenApi() {
        return new OpenAPI()
                .info(new Info()
                        .title("University Management System API")
                        .version("1.0.0")
                        .description("""
                                CSE 3230 Lab 4. Log in either way:
                                1. Click **Authorize** and enter a username/password (HTTP Basic),
or
                                2. Call **POST /api/auth/login**; the browser keeps the session
cookie.

                                Seeded admin: `admin` / `admin123`."""))
                .components(new Components().addSecuritySchemes(BASIC_AUTH,
                        new SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("basic")))
                .addSecurityItem(new SecurityRequirement().addList(BASIC_AUTH))
                .path("/api/auth/logout", new PathItem().post(new Operation()
                        .tags(List.of("Authentication"))
                        .summary("Log out: invalidates the session and deletes the JSESSIONID
cookie")
                        .responses(new ApiResponses().addApiResponse("200",
                                new ApiResponse().description("Logged out successfully")))));
    }
}
```

- Without this class springdoc already works: it finds all controller endpoints by itself. This bean adds three things on top.
- `.info(...)`: the title, version and description at the top of the page (Markdown: `**bold**`, `` `code` ``).
- `.components(...addSecuritySchemes("basicAuth", HTTP + "basic"))`: declares that the API accepts HTTP Basic. This is what makes the **Authorize** button appear. `.addSecurityItem(...)` applies

it to every endpoint (the lock icons).

- `.path("/api/auth/logout", ...)`: logout is handled by Spring Security's `LogoutFilter`, not by a controller method, so the scanner cannot see it. We describe it by hand so it shows up as a button too.
- The page is documentation only: it calls the same endpoints, through the same security rules, as Postman or curl. Swagger UI does not bypass anything.

18.3 Using Postman instead

- Session way: `POST http://localhost:8080/api/auth/login`, Body → raw → JSON `{"username":"admin","password":"admin123"}`. Postman stores the `JSESSIONID` cookie and sends it on later requests automatically.
- Basic way: on any request, Authorization tab → Basic Auth → username/password.
- For POST/PUT set header `Content-Type: application/json` (Postman does it when you choose raw → JSON).

19. Likely viva questions and answers

Q1. What is REST? Is your API RESTful?

REST is an architectural style: data is exposed as **resources** identified by URLs, manipulated with standard HTTP methods (GET/POST/PUT/DELETE), exchanged as representations (JSON), and results are signalled by status codes. Our API follows this: `/api/teachers/{id}` with the right methods and codes. One honest caveat: REST prefers stateless servers, and session login keeps login state on the server; token (JWT) auth would be fully stateless. HTTP Basic, which we also support, is stateless.

Q2. Difference between PUT, PATCH and POST?

POST creates a new resource (not idempotent). PUT replaces the resource at a known URL with the sent representation (idempotent). PATCH changes only the sent fields. We use PUT with the full object; password is the one optional field.

Q3. Why return 201 and a Location header on create? Why 204 on delete?

201 tells the client something new was created, and `Location` tells it the new resource's URL. After a delete there is nothing to send back, and 204 means "success, empty body".

Q4. Difference between 401 and 403?

401 Unauthorized = not authenticated, the server doesn't know who you are (no/expired session, wrong password). 403 Forbidden = authenticated, but your role isn't allowed (teacher calling `/api/students`). Spring decides in `ExceptionTranslationFilter`: anonymous → entry point (401), real user → access denied handler (403).

Q5. What does @SpringBootApplication do?

It is `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`: the class can define beans, Spring Boot configures libraries found on the classpath, and all our annotated classes under `com.university.ums` are discovered.

Q6. What is dependency injection? Why constructor injection?

Objects receive their dependencies from the Spring container instead of creating them. Constructor injection makes dependencies explicit and `final`, objects are always fully initialised, and in tests you can pass fakes easily.

Q7. JPA vs Hibernate vs Spring Data JPA?

JPA is the specification (annotations and interfaces). Hibernate is the implementation that generates SQL. Spring Data JPA sits on top and generates repository implementations from interfaces and method names.

Q8. How does findByUsername work without any code?

Spring Data parses the method name (`find…By` + property `Username`) and generates the query `WHERE username = ?`, then creates a proxy class implementing the interface at startup.

Q9. Explain your database design. Why inheritance?

Abstract `AppUser` with JOINED inheritance: a `users` table for common fields plus `admins`, `teachers`, `students` tables sharing the same id (FK to users). One users table enforces unique usernames/emails across all roles and lets login search one place; child tables keep role-specific columns NOT NULL. (Section 6 compares the alternatives.)

Q10. What is a DTO? Why not return the entity?

A plain object that defines the API's JSON. Returning the entity would expose the password hash, would let clients set fields like id/role (mass assignment), and ties the API to the database schema.

Q11. How does validation work?

Annotations like `@NotBlank`, `@Email` on the request record; `@Valid` on the controller parameter makes Spring validate before the method runs. Failures throw `MethodArgumentNotValidException`, which our `GlobalExceptionHandler` converts to 400 with a `fieldErrors` map.

Q12. What does @Transactional do? What if an exception occurs halfway?

The method runs in one database transaction: commit on success, rollback on a `RuntimeException`. For example, if the INSERT into `teachers` failed after the INSERT into `users`, both would be undone, never leaving half a teacher.

Q13. How are passwords stored? Can you show me a user's password?

As BCrypt hashes (`$2a$10$…`, 60 chars). Nobody, including the admin or developer, can get the original password back; hashing is one-way. Login works by hashing the typed password with the stored salt and comparing.

Q14. What is a salt? Why BCrypt and not MD5/SHA-256?

A salt is random data mixed into each hash so equal passwords give different hashes and pre-computed tables don't work. BCrypt is deliberately slow (cost factor 10 = 1024 rounds) and has the salt built in; MD5/SHA are fast, so attackers can test billions of guesses per second.

Q15. Walk me through what happens when I log in.

See 13.4: login endpoint → AuthenticationManager → DaoAuthenticationProvider → our UserDetailsService loads the user → BCrypt `matches` → authenticated token with ROLE_X → new session id → SecurityContext saved in the session → 200 + JSESSIONID cookie.

Q16. How does the server know who I am on the next request?

The browser/client sends the `JSESSIONID` cookie. `SecurityContextHolderFilter` uses it to find the server-side session and loads the SecurityContext saved at login.

Q17. How does logout work? There is no logout method in your controller.

Spring Security's `LogoutFilter` is configured for `POST /api/auth/logout`. It invalidates the session, clears the context, deletes the cookie and calls our success handler that returns JSON. Since the session is destroyed on the server, an old cookie no longer works.

Q18. Where is role-based access control implemented? Show me why a teacher can't access students.

In `SecurityConfig.authorizeHttpRequests`: `/api/students/**` → `hasAnyRole("ADMIN","STUDENT")`. A teacher has only ROLE_TEACHER, so AuthorizationFilter denies → 403 before the controller is reached. The test `teacherCanManageTeachersOnly` proves it.

Q19. hasRole vs hasAuthority?

`hasRole("ADMIN")` checks for authority `ROLE_ADMIN` (adds the prefix). `hasAuthority("ROLE_ADMIN")` is the same check written out. Authorities can also be fine-grained permissions like `student:delete`.

Q20. Does the order of the authorization rules matter?

Yes, first match wins. A broad rule before a specific one would hide it. `anyRequest()` must be last; Spring refuses to start otherwise.

Q21. Why did you disable CSRF? Isn't that insecure?

CSRF tokens are designed for HTML forms; this is a JSON API for Postman/curl/JS clients. Protection is kept through `SameSite=Lax` on the session cookie (browsers won't send it on cross-site POST/PUT/DELETE) and because no GET request changes data. Basic-auth clients are immune to CSRF. For a future browser front-end, re-enabling CSRF with a cookie token repository would be better.

Q22. What is session fixation and how do you prevent it?

An attacker makes the victim use a session id the attacker knows, waits for the victim to log in, then uses that id. We call `changeSessionId()` at login, so the authenticated session always has a fresh id.

Q23. Is HTTP Basic secure?

The credentials are only Base64-encoded, not encrypted, and sent on every request. It's secure only over HTTPS. We support it for convenience in testing; in production the app would sit behind HTTPS.

Q24. How is the first admin created? What if I restart?

`DataInitializer` (a `CommandLineRunner`) runs at startup and creates an admin from `app.admin.*` settings only if no admin exists. Restarting does nothing.

Q25. Why can't the last admin be deleted?

Only admins can create admins. Deleting the last one would make admin management impossible. The service returns 409 Conflict.

Q26. What happens if two people register the same username at the same time?

Both Java checks might pass, but the database UNIQUE constraint rejects the second INSERT. The resulting `DataIntegrityViolationException` is mapped to 409 by the global handler.

Q27. Why are usernames lower-cased?

So "Rahim" and "rahim" can't be two different accounts, and behaviour is identical on MySQL (case-insensitive collation) and H2 (case-sensitive). Login normalises too, so users can type any case.

Q28. What is Docker? Image vs container?

Docker packages an application with everything it needs into an image; a container is a running instance of that image, isolated from the host. Same image → same behaviour on every machine.

Q29. Why a multi-stage Dockerfile?

The build needs Maven and a JDK; running needs only a JRE and the jar. Stage 2 copies only the jar, so the final image is smaller and contains no build tools or source code.

Q30. How does the app find the database inside Docker?

Compose creates a private network and a DNS entry for each service name. The URL `jdbc:mysql://db:3306/university_db` uses the service name `db`. `localhost` inside the app container would be the app container itself.

Q31. What do depends_on and healthcheck do? Why not just depends_on?

Plain `depends_on` only waits until the db container has started, but MySQL needs several seconds more to accept connections, so Spring would fail. With `condition: service_healthy`, compose waits until `mysqladmin ping` succeeds.

Q32. Where is the data stored? What happens on docker compose down?

In the named volume `db-data` mounted at `/var/lib/mysql`. `down` removes containers but keeps the volume; `down -v` deletes it.

Q33. Why not publish MySQL's port 3306?

Only the app needs the database, and it reaches it over the internal network. Not publishing the port means nothing outside Docker can connect to MySQL.

Q34. How do environment variables in compose reach Spring?

Spring Boot's relaxed binding maps `SPRING_DATASOURCE_URL` to `spring.datasource.url`, and environment variables override application.yml.

Q35. Why H2 for tests and MySQL for running?

Tests should be fast and need nothing installed; H2 runs in memory in MySQL compatibility mode. The real app uses MySQL. (Testcontainers could run tests against a real MySQL container — a stricter alternative.)

Q36. Session vs JWT – which is better?

Sessions: simple, built into Spring, logout is instant (session destroyed), but the server stores state and scaling to several servers needs shared session storage. JWT: stateless and good for mobile/microservices, but needs extra code and you can't easily revoke a token before it expires. For a single-server lab app, sessions are the simpler and safer choice.

Q37. Why does @RestControllerAdvice not handle 401/403?

Those happen in Spring Security filters before Spring MVC; the advice only sees exceptions from controllers. We use `AuthenticationEntryPoint` and `AccessDeniedHandler` implementations instead.

Q38. What is Jackson? What are records?

Jackson is the JSON library Spring uses to convert JSON ↔ Java objects. Records are compact immutable data classes (Java 16+) with auto-generated constructor, accessors, equals/hashCode/toString.

Q39. What does ddl-auto: update do? Would you use it in production?

It creates/alters tables from the entities at startup. In production, schema changes should be versioned and reviewed, using Flyway or Liquibase migrations, with `ddl-auto: validate`.

Q40. What if teachers should only edit their own profile?

Add method security: `@EnableMethodSecurity`, then in the service compare the logged-in username with the target record (or `@PreAuthorize("hasRole('ADMIN') or @teacherService.isOwner(#id, authentication.name)")`). The assignment says teachers manage teachers, so full CRUD was implemented as written.

Q41. Why is Spring Security's explicit saveContext needed?

Since Spring Security 6, `SecurityContextHolderFilter` only loads the context; it no longer saves changes automatically at the end of the request. Code that logs a user in manually must call `securityContextRepository.saveContext`.

Q42. What is Swagger / OpenAPI? Isn't a public docs page a security risk?

OpenAPI is a standard JSON/YAML format that describes a REST API; Swagger UI is a web page that renders it and lets you send requests. springdoc generates the description from our controllers. Only the documentation is public; every request made from the page goes through the same authentication and RBAC rules. In production you would often disable it (`springdoc.swagger-ui.enabled=false`, `springdoc.api-docs.enabled=false`) or restrict it to admins, because it reveals the API's structure.

Q43. What does saveAndFlush do differently from save?

`save` may delay the SQL until commit. `saveAndFlush` sends it immediately, which triggers `@PreUpdate` so the response contains the new `updatedAt`.

20. Known limitations and possible improvements

Be ready to mention these yourself; it shows you understand the design.

Limitation	Improvement
A teacher can edit or delete *any* teacher; a student any student (the assignment's literal wording)	Ownership checks with method security (Q40)
If a user is deleted or their password changes, their existing session keeps working until logout/timeout	Spring Session with a session registry to expire that user's sessions, or reload the user on each request
`GET /api/students` returns all rows	Pagination and search: `Pageable` parameter → `?page=0&size=20&sort=name`
Schema created by `ddl-auto: update`	Flyway migration scripts
No HTTPS	A reverse proxy (e.g. Nginx/Caddy) with TLS in front of the app
No brute-force protection on login	Rate limiting / account lock after N failures
CSRF disabled (SameSite Lax instead)	Enable CSRF with `CookieCsrfTokenRepository` if a browser front-end is added
Default demo credentials in compose	Required `.env` values / Docker secrets
Docker image based on full Ubuntu JRE	Alpine or distroless base, or a custom JRE with `jlink`, for a smaller image
No health endpoint for the app container	Spring Boot Actuator `/actuator/health` + compose healthcheck
Swagger UI and the API description are public	Disable them in a production profile, or require ADMIN for `/swagger-ui/**`
Tests use H2, not MySQL	Testcontainers with a real MySQL

Remaining source files (for completeness)

src/main/java/com/university/ums/controller/AdminController.java

```java
package com.university.ums.controller;

import com.university.ums.dto.AdminRequest;
import com.university.ums.dto.AdminResponse;
import com.university.ums.service.AdminService;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;
```

```java
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;

/**
 * CRUD endpoints for admins. Who may call them is decided in SecurityConfig
 * (/api/admins/** rule), not here.
 */
@Tag(name = "Admins", description = "Manage admins (ADMIN only)")
@RestController
@RequestMapping("/api/admins")
public class AdminController {

    private final AdminService adminService;

    public AdminController(AdminService adminService) {
        this.adminService = adminService;
    }

    // GET /api/admins -> 200 OK + list
    @GetMapping
    public List<AdminResponse> getAll() {
        return adminService.findAll();
    }

    // GET /api/admins/{id} -> 200 OK, or 404 Not Found
    @GetMapping("/{id}")
    public AdminResponse getById(@PathVariable Long id) {
        return adminService.findById(id);
    }

    // POST /api/admins -> 201 Created + Location header pointing at the new resource
    @PostMapping
    public ResponseEntity<AdminResponse> create(@Valid @RequestBody AdminRequest request) {
        AdminResponse created = adminService.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}")
                .buildAndExpand(created.id())
                .toUri();
        return ResponseEntity.created(location).body(created);
    }

    // PUT /api/admins/{id} -> 200 OK + updated resource
    @PutMapping("/{id}")
    public AdminResponse update(@PathVariable Long id, @Valid @RequestBody AdminRequest request) {
        return adminService.update(id, request);
    }

    // DELETE /api/admins/{id} -> 204 No Content
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        adminService.delete(id);
    }
}
```

src/main/java/com/university/ums/controller/StudentController.java

```java
package com.university.ums.controller;

import com.university.ums.dto.StudentRequest;
import com.university.ums.dto.StudentResponse;
import com.university.ums.service.StudentService;
import io.swagger.v3.oas.annotations.tags.Tag;
```

```java
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;

/**
 * CRUD endpoints for students. Who may call them is decided in SecurityConfig
 * (/api/students/** rule), not here.
 */
@Tag(name = "Students", description = "Manage students (ADMIN or STUDENT)")
@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    // GET /api/students -> 200 OK + list
    @GetMapping
    public List<StudentResponse> getAll() {
        return studentService.findAll();
    }

    // GET /api/students/{id} -> 200 OK, or 404 Not Found
    @GetMapping("/{id}")
    public StudentResponse getById(@PathVariable Long id) {
        return studentService.findById(id);
    }

    // POST /api/students -> 201 Created + Location header pointing at the new resource
    @PostMapping
    public ResponseEntity<StudentResponse> create(@Valid @RequestBody StudentRequest request) {
        StudentResponse created = studentService.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}")
                .buildAndExpand(created.id())
                .toUri();
        return ResponseEntity.created(location).body(created);
    }

    // PUT /api/students/{id} -> 200 OK + updated resource
    @PutMapping("/{id}")
    public StudentResponse update(@PathVariable Long id, @Valid @RequestBody StudentRequest
request) {
        return studentService.update(id, request);
    }

    // DELETE /api/students/{id} -> 204 No Content
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
```

```
        studentService.delete(id);
    }
}
```

src/main/java/com/university/ums/dto/AdminRequest.java

```
package com.university.ums.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

/**
 * JSON body for POST /api/admins and PUT /api/admins/{id}.
 * password is required on create, optional on update (omit it to keep the old one).
 */
public record AdminRequest(
        @NotBlank @Size(max = 100) String name,
        @NotBlank @Email @Size(max = 100) String email,
        @Size(max = 20) String phone,
        @NotBlank @Size(min = 3, max = 50)
        @Pattern(regexp = "^[A-Za-z0-9._-]+$", message = "may contain only letters, digits, '.',
'_' and '-'")
        String username,
        @Size(min = 6, max = 72) String password
) {
}
```

src/main/java/com/university/ums/dto/AdminResponse.java

```
package com.university.ums.dto;

import com.university.ums.model.Admin;
import com.university.ums.model.Role;

import java.time.Instant;

/** What the API sends back. No password field, so the hash can never leak. */
public record AdminResponse(
        Long id,
        String name,
        String email,
        String phone,
        String username,
        Role role,
        Instant createdAt,
        Instant updatedAt
) {
    public static AdminResponse from(Admin admin) {
        return new AdminResponse(
                admin.getId(),
                admin.getName(),
                admin.getEmail(),
                admin.getPhone(),
                admin.getUsername(),
                admin.getRole(),
                admin.getCreatedAt(),
                admin.getUpdatedAt());
    }
}
```

src/main/java/com/university/ums/dto/StudentResponse.java

```java
package com.university.ums.dto;

import com.university.ums.model.Role;
import com.university.ums.model.Student;

import java.time.Instant;

public record StudentResponse(
        Long id,
        String name,
        String email,
        String phone,
        String username,
        Role role,
        String studentId,
        String department,
        Integer batch,
        Instant createdAt,
        Instant updatedAt
) {
    public static StudentResponse from(Student student) {
        return new StudentResponse(
                student.getId(),
                student.getName(),
                student.getEmail(),
                student.getPhone(),
                student.getUsername(),
                student.getRole(),
                student.getStudentId(),
                student.getDepartment(),
                student.getBatch(),
                student.getCreatedAt(),
                student.getUpdatedAt());
    }
}
```

src/main/java/com/university/ums/repository/AdminRepository.java

```java
package com.university.ums.repository;

import com.university.ums.model.Admin;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AdminRepository extends JpaRepository<Admin, Long> {
}
```

src/main/java/com/university/ums/exception/ConflictException.java

```java
package com.university.ums.exception;

/** Mapped to 409 Conflict: duplicate username/email/studentId, or deleting the last admin. */
public class ConflictException extends RuntimeException {

    public ConflictException(String message) {
        super(message);
    }
}
```

src/main/java/com/university/ums/exception/BadRequestException.java

```java
package com.university.ums.exception;

/** Mapped to 400 Bad Request for rule violations that annotations cannot express. */
public class BadRequestException extends RuntimeException {

    public BadRequestException(String message) {
        super(message);
    }
}
```

src/test/resources/application.yml

```yaml
# Tests use an in-memory H2 database so they need no MySQL.
spring:
  datasource:
    url: jdbc:h2:mem:testdb;MODE=MySQL;DB_CLOSE_DELAY=-1
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
    open-in-view: false

app:
  admin:
    username: admin
    password: admin123
    email: admin@university.edu
  demo-data: false   # tests expect only the seeded admin
```